Dossier de Projet : Music Wall

Titre RNCP Niveau 6 - RNCP37873

Titre Professionnel Concepteur Développeur d'Applications

Nom et prénom : Gudumac Janna

Année : 2026

École : DesCodeuses

Promo : Danielle

Table des matières

I. Personal presentation 4
II. Présentation du projet 4
III - Cahier des charges 4
A - Les objectifs 4
B - Cibles 5
C - Exigences fonctionnelles 5
1 - Utilisateur non authentifié 5
2 - Utilisateur authentifié 5
3 - Propriétaire d'un mur 6
4 - Membre d'un mur 6
5 - Catalogue musical 6
6 - Confidentialité 7
7 - Droits d'accès 7
8 - Authentification 7
D - Exigences et choix techniques 8
1 - Exigences 8
2 - Choix techniques 8
E - Définition du MVP 9
F - Contraintes du projet 10
IV - Méthodologie et organisation 10
A - Product Backlog 11
B - User Stories 11
V - Conception de l'interface graphique 12
A - Zoning 12
B - Wireframe 14
C - Charte graphique 15
D - Maquettage 17
E - Responsive design 18
VI - Conception de la base de données 20
A - Modèle Conceptuel de Données – MCD 20
B - Modèle Logique de Données – MLD 21
C - Modèle Physique de Données - MPD 22
VII - Conception de l'application 24
A - Use Case 24
B - Diagramme de classes 25
C - Diagramme de séquence 26

VIII. Architecture logicielle 26
IX. Architecture multicouche 27
A - Couche de présentation 27
B - Couche de traitement et logique métier 28
C - Couche d'accès aux données 28
D - Communication entre les couches 28
X. Sécurité 29
A. Authentification / autorisation 29
B. XSS 30
C. Injection SQL 30
D. CSRF 31
E. JWT 31
F. BCrypt 32
G. CORS, validation et security headers 32
1 - CORS 33
2 - Validation 33
3 - Security headers 33
XI. Politique de tests 34
A. Tests unitaires 34
B. Tests d'intégration 34
C. Tests manuels de l'API avec Postman 35
XII. Déploiement 36
A. Docker 36
A. Docker Compose / Nginx 37
C. Production Netlify / Render 38
XIII. Veille technologique 39
XIV. Difficultés rencontrées 40
A - Gestion des droits d'accès 40
B - Gestion du catalogue musical 40
C - Modélisation des relations 40
D - Déploiement de l'application 40
XV. Conclusion / améliorations futures 41
XVI. Remerciements 41

I. Personal presentation

My name is Janna Gudumac. I have a background in English language, literature and translation, and I later decided to transition into the field of information technology. I completed a full-stack development training programme at DesCodeuses, where I worked with technologies such as Java, Spring Boot, Angular and relational databases.

I am currently continuing my studies in computer science at the University of Montpellier. During my training, I became particularly interested in the architecture of web applications, backend development and the way different layers of an application communicate with each other.

For my RNCP Level 6 certification project, I developed **Music Wall**, a full-stack web application designed to let users create shared musical spaces, organise music into sections and discover artists, albums and tracks through a catalogue.

II. Présentation du projet

Music Wall, l'idée de murs musicaux partagés

Music Wall est une application web full-stack conçue autour de l'idée d'un espace musical partagé. Elle permet à plusieurs utilisateurs de créer et d'organiser ensemble des sélections musicales sous la forme de « murs ». Chaque mur peut être divisé en différentes sections afin de regrouper des morceaux ou des albums selon un thème, un genre, une ambiance ou tout autre critère choisi par les utilisateurs.

L'objectif du projet est de proposer une manière plus visuelle et collaborative de partager des découvertes musicales qu'une simple playlist. Le propriétaire d'un mur peut y ajouter d'autres utilisateurs, créer des sections et y intégrer des morceaux ou des albums provenant du catalogue de l'application. Les membres autorisés peuvent ensuite consulter le contenu du mur et suivre l'écoute des morceaux grâce aux statuts « To listen » et « Listened ».

L'application dispose également d'un catalogue musical permettant de rechercher et de consulter des artistes, des albums et des morceaux. Les utilisateurs disposent d'un compte personnel avec un profil, tandis que l'accès aux murs partagés est contrôlé selon les droits du propriétaire et des membres.

Music Wall a ainsi été conçu comme une application combinant découverte musicale, organisation et collaboration, tout en mettant en pratique les différentes composantes d'une application web moderne : interface responsive, API REST, persistance des données et sécurisation des accès.

III - Cahier des charges

A - Les objectifs

Music Wall a pour objectif principal de proposer un espace permettant à plusieurs utilisateurs d'organiser et de partager des découvertes musicales de manière collaborative.

L'application permet de créer des murs musicaux personnalisés, de les diviser en sections et d'y ajouter des morceaux ou des albums issus d'un catalogue musical. Chaque mur peut ainsi être organisé selon les besoins de ses utilisateurs : genre musical, ambiance, période, recommandations ou tout autre thème.

Un autre objectif est de faciliter le suivi de l'écoute. Les morceaux ajoutés à un mur peuvent être associés à un statut « To listen » ou « Listened », permettant aux utilisateurs de distinguer rapidement les contenus restant à découvrir de ceux déjà écoutés.

Le projet vise également à proposer une navigation simple entre les différents espaces de l'application : dashboard, murs musicaux, catalogue et profil utilisateur.

D'un point de vue technique, Music Wall a pour objectif de mettre en œuvre une application web full-stack organisée en couches, avec une interface responsive, une API REST, une base de données relationnelle et un système d'authentification et d'autorisation sécurisé.

B - Cibles

Music Wall s'adresse principalement aux personnes qui aiment découvrir, organiser et partager de la musique.

L'application peut notamment être utilisée par des groupes d'amis souhaitant créer ensemble des sélections musicales, recommander des morceaux ou des albums et conserver une trace des contenus qu'ils souhaitent écouter.

Elle peut également convenir à un utilisateur individuel souhaitant organiser ses découvertes musicales dans différents murs et sections plutôt que dans une simple liste unique.

L'application est accessible depuis un navigateur web et son interface est adaptée aux écrans d'ordinateur, de tablette et de smartphone.

C - Exigences fonctionnelles

1 - Utilisateur non authentifié

Un utilisateur non authentifié doit pouvoir :

• créer un compte à l'aide d'un nom d'utilisateur et d'un mot de passe ;
• se connecter à un compte existant ;
• recevoir un message d'erreur en cas d'identifiants incorrects ;
• être redirigé vers l'application après une authentification réussie.

Les pages nécessitant une authentification ne doivent pas être accessibles à un utilisateur non connecté.

2 - Utilisateur authentifié

Une fois connecté, l'utilisateur doit pouvoir :

• accéder à son dashboard ;
• consulter les murs musicaux auxquels il a accès ;
• créer un nouveau mur musical ;
• consulter le catalogue musical ;
• rechercher des artistes, des albums et des morceaux ;
• consulter les informations détaillées d'un artiste, d'un album ou d'un morceau ;
• consulter et modifier son profil ;
• ajouter ou modifier une photo de profil ;
• modifier sa biographie ;

• changer son mot de passe ;
• se déconnecter.

3 - Propriétaire d'un mur

Le créateur d'un mur en devient le propriétaire.

Le propriétaire doit pouvoir :

• modifier le nom du mur ;
• personnaliser son apparence ;
• modifier la couleur ou le fond visuel du mur ;
• supprimer le mur ;
• ajouter des utilisateurs enregistrés comme membres ;
• retirer des membres du mur ;
• créer des sections ;
• modifier les sections ;
• supprimer les sections ;
• ajouter des morceaux ou des albums dans les sections ;
• supprimer des éléments musicaux ;
• modifier leur statut d'écoute.

Le propriétaire conserve les droits de gestion du mur.

4 - Membre d'un mur

Un utilisateur ajouté comme membre doit pouvoir accéder au mur partagé et consulter son contenu.

Un membre peut également créer, modifier et supprimer des sections, ajouter ou supprimer des éléments musicaux et modifier leur statut d'écoute.

Les opérations de gestion du mur lui-même, telles que la modification du nom, la personnalisation de l'apparence, la suppression du mur ou la gestion des membres, restent réservées au propriétaire.

Un utilisateur qui n'est ni propriétaire ni membre d'un mur ne doit pas pouvoir y accéder.

5 - Catalogue musical

Le catalogue doit permettre de consulter les données musicales stockées dans la base de données de l'application.

Il doit contenir :

• des artistes ;
• des albums ;
• des morceaux ;
• des genres musicaux.

L'utilisateur doit pouvoir effectuer une recherche et filtrer les résultats par type : tous les résultats, artistes, albums ou morceaux.

Les pages de détail doivent permettre de consulter les principales informations disponibles pour chaque élément du catalogue.

6 - Confidentialité

La protection des données utilisateur et le contrôle de l'accès aux murs constituent des exigences importantes du projet.

Les mots de passe ne doivent jamais être enregistrés en clair dans la base de données. Ils doivent être transformés en hash avant leur enregistrement.

L'accès aux murs doit être contrôlé côté backend. Il ne suffit pas de masquer une fonctionnalité dans l'interface Angular : le serveur doit vérifier que l'utilisateur authentifié possède réellement les droits nécessaires.

Les requêtes authentifiées vers l'API doivent inclure un token JWT.

Dans l'interface Angular, les pages du catalogue et des profils sont accessibles après authentification. Au niveau de l'API, certaines opérations de lecture du catalogue et des profils sont publiques, tandis que les opérations de modification ainsi que l'accès aux murs musicaux nécessitent une authentification.

Les données sensibles de configuration, telles que la clé utilisée pour signer les tokens ou les mots de passe de la base de données, ne doivent pas être intégrées directement au code source public.

7 - Droits d'accès

L'application possède une autorité technique globale USER, utilisée par Spring Security, mais ne comporte pas de rôle ADMIN.

Les autorisations métier dépendent principalement de la relation entre l'utilisateur et le mur : propriétaire, membre ou utilisateur extérieur.

Un utilisateur authentifié peut accéder aux fonctionnalités générales de l'application et gérer son propre profil.

Pour un mur musical, trois situations sont distinguées :

- propriétaire : peut modifier le mur, personnaliser son apparence, le supprimer, ajouter ou retirer des membres et gérer son contenu ;
- membre : peut accéder au mur et gérer les sections, les éléments musicaux et leur statut d'écoute ;
- autre utilisateur : ne peut pas accéder au mur s'il n'en est ni propriétaire ni membre.

Les contrôles d'autorisation sont effectués côté backend avant l'exécution des opérations protégées.

8 - Authentification

L'authentification de Music Wall repose sur Spring Security et les JSON Web Tokens. Lors de l'inscription, le backend valide les informations reçues, vérifie que le nom d'utilisateur est disponible et encode le mot de passe avec BCrypt avant de créer l'utilisateur dans la base de données.

La longueur minimale demandée lors de la création ou du changement d'un mot de passe est de huit caractères. Lors de la connexion, Spring Security vérifie les identifiants. Si l'authentification réussit, le backend génère un JWT contenant l'identité de l'utilisateur.

Le frontend conserve le token et l'ajoute automatiquement aux requêtes protégées dans le header HTTP : Authorization: Bearer <token>

Côté backend, un filtre JWT intercepte les requêtes, vérifie la validité du token et charge l'utilisateur authentifié dans le SecurityContext de Spring Security. Les endpoints protégés ne sont accessibles que lorsqu'une authentification valide est présente.

D - Exigences et choix techniques

1 - Exigences

L'application doit être utilisable sur les navigateurs web modernes, notamment Google Chrome, Mozilla Firefox et Microsoft Edge. L'interface doit être responsive et s'adapter aux différentes tailles d'écran.

Le concept principal de Music Wall étant particulièrement adapté à un affichage large, l'interface a d'abord été structurée pour desktop, puis adaptée progressivement aux écrans plus petits.

Des adaptations spécifiques ont été mises en place pour les smartphones : navigation inférieure fixe, réorganisation des actions, retour à la ligne des contenus longs, adaptation des cartes et réduction de certains espaces et éléments visuels.

L'interface doit conserver une bonne lisibilité, éviter les débordements horizontaux et laisser les principales actions accessibles sur les petits écrans. Les formulaires doivent proposer des labels et des messages d'erreur compréhensibles.

2 - Choix techniques

Le frontend de Music Wall est développé avec Angular et TypeScript.

Angular a été choisi pour son organisation par composants, son système de routing, ses services HTTP, ses formulaires, ses guards et ses interceptors. TypeScript apporte un typage statique qui facilite la détection d'erreurs pendant le développement.

Le style de l'interface est principalement réalisé avec des feuilles CSS propres aux composants. Ce choix permet de personnaliser précisément l'identité visuelle de Music Wall et d'adapter les différents composants aux besoins responsive de l'application.

Le backend est développé en Java avec Spring Boot. Spring Boot permet de construire une API REST et fournit une intégration avec Spring Web, Spring Data JPA et Spring Security.

Le backend est organisé en plusieurs couches séparant les responsabilités :

- Controller : réception des requêtes HTTP ;
- Service : logique métier ;
- Repository : accès aux données ;
- Entity : représentation des données persistantes ;
- DTO : échanges de données avec l'API.

PostgreSQL est utilisé comme système de gestion de base de données relationnelle.

La persistance des données est assurée avec Spring Data JPA et Hibernate. Les entités utilisent principalement les annotations JPA ManyToOne et ManyToMany. La base de données contient également des relations de type un-à-plusieurs, représentées côté Java depuis l'entité enfant avec ManyToOne.

La communication entre Angular et Spring Boot est réalisée via une API REST utilisant le format JSON et les méthodes HTTP GET, POST, PUT et DELETE.

La sécurité repose notamment sur Spring Security, JWT et BCrypt.

Docker et Docker Compose sont utilisés afin de pouvoir lancer localement l'ensemble de l'application dans un environnement reproductible comprenant le frontend, le backend et PostgreSQL.

Dans l'environnement Docker, Angular est compilé puis servi par Nginx. Nginx joue également le rôle de reverse proxy en transférant les requêtes /api vers le backend Spring Boot.

E - Définition du MVP

Le MVP de Music Wall comprend les fonctionnalités suivantes :

• inscription et connexion ;
• authentification JWT ;
• hash des mots de passe avec BCrypt ;
• profil utilisateur avec avatar et biographie ;
• changement de mot de passe ;
• création, modification et suppression de murs musicaux ;
• personnalisation visuelle des murs ;
• ajout et suppression de membres ;
• création, modification et suppression de sections ;
• ajout de morceaux et d'albums à une section ;
• suppression d'éléments musicaux ;
• statuts « To listen » et « Listened » ;
• catalogue local d'artistes, d'albums, de morceaux et de genres ;
• recherche dans le catalogue ;
• contrôle des droits propriétaire/membre ;
• interface responsive ;
• API REST ;
• persistance PostgreSQL ;
• environnement Docker Compose.

Certaines fonctionnalités envisagées lors des premières réflexions ont été reportées afin de conserver un MVP cohérent et réalisable.

Les évolutions possibles pour une version ultérieure comprennent notamment :

• système d'amis ;
• invitations avec acceptation ou refus avant de rejoindre un mur ;
• gestion plus avancée des rôles des membres ;
• favoris ;
• statistiques musicales ;
• historique de navigation plus avancé ;

• intégration d’autres API musicales ou événementielles ;
• fonctionnalités collaboratives supplémentaires.

F - Contraintes du projet

Le développement de Music Wall a été réalisé dans le cadre d’un projet de certification avec une durée limitée.

La principale contrainte a été de définir un périmètre suffisamment complet pour démontrer les compétences attendues tout en évitant une complexité excessive.

Certaines fonctionnalités initialement envisagées ont donc été simplifiées ou reportées à une version ultérieure.

Une autre contrainte importante concernait l’interface responsive. La structure visuelle des murs musicaux étant naturellement plus lisible sur un écran large, plusieurs composants ont dû être réorganisés spécifiquement pour les smartphones afin de conserver une navigation claire et des actions facilement accessibles.

Le catalogue musical représentait également une contrainte particulière. Les données utilisées par l’application sont stockées localement dans PostgreSQL afin que le fonctionnement principal de Music Wall ne dépende pas d’une API musicale externe en temps réel.

IV - Méthodologie et organisation

Le développement de Music Wall a été réalisé dans le cadre d'un projet individuel. J'ai adopté une démarche itérative inspirée des principes agiles, sans appliquer de manière stricte un framework comme Scrum. Le développement a été organisé progressivement autour des fonctionnalités prioritaires du MVP, avec des ajustements du périmètre en fonction des difficultés rencontrées et du temps disponible.

Dans un premier temps, j’ai défini les fonctionnalités essentielles du MVP : authentification, gestion des utilisateurs, création de murs musicaux, gestion des membres et des sections, ajout de contenus musicaux, catalogue et sécurisation des accès. Cette définition m’a permis de concentrer le développement sur les fonctionnalités nécessaires au fonctionnement cohérent de l’application.

Le projet a ensuite évolué par étapes. Chaque fonctionnalité a été développée puis vérifiée avant de passer à la suivante. Lorsque certaines fonctionnalités se sont révélées trop complexes par rapport au temps disponible ou peu indispensables au MVP, leur périmètre a été simplifié ou leur développement reporté à une version ultérieure. C’est notamment le cas du système d’amis, des invitations avec acceptation, des favoris et de certaines statistiques musicales.

Cette organisation progressive m’a également permis d’améliorer l’interface à partir du fonctionnement réel de l’application. Après avoir réalisé les principales fonctionnalités desktop, j’ai adapté les différents écrans aux smartphones en vérifiant notamment la navigation, le comportement des cartes, le retour à la ligne des textes longs, la disposition des boutons et l’absence de débordement horizontal.

Le projet a été organisé autour de plusieurs parties techniques distinctes :

• frontend Angular pour l’interface utilisateur ;
• backend Spring Boot pour l’API REST et la logique métier ;
• PostgreSQL pour la persistance des données ;
• scripts et données d’initialisation du catalogue musical ;
• configuration Docker pour l’exécution de l’ensemble de l’application ;
• documentation technique et éléments nécessaires à la préparation de la soutenance.

Pour le versioning, j'ai utilisé Git. Les modifications sont enregistrées progressivement dans le dépôt afin de conserver un historique du développement et de pouvoir identifier les changements apportés au projet. L'utilisation d'un système de versioning permet également de revenir à une version précédente en cas de problème et d'éviter de modifier directement une version fonctionnelle sans possibilité de restauration.

Une attention particulière a été portée à la vérification du projet après les modifications importantes. Les builds du frontend et du backend ainsi que les tests automatisés ont été exécutés régulièrement afin de détecter rapidement les régressions. Le fonctionnement de l'environnement Docker a également été vérifié avec le frontend, le backend, PostgreSQL et l'initialisation du catalogue.

Cette méthode de travail m'a permis de faire évoluer progressivement Music Wall tout en maintenant un périmètre compatible avec les contraintes de temps du projet et en conservant une version fonctionnelle de l'application à chaque étape importante.

A - Product Backlog

Afin de structurer les principales fonctionnalités du projet et de distinguer les éléments indispensables au MVP des évolutions secondaires, j'ai regroupé les besoins principaux de Music Wall dans un Product Backlog. Les fonctionnalités nécessaires au fonctionnement de l'application ont été traitées en priorité, tandis que certaines fonctionnalités plus complexes ont été reportées à une version ultérieure.

Fonctionnalité	**Priorité**	**Statut**
Inscription et connexion	Haute	Terminé
Authentification JWT	Haute	Terminé
Création et gestion des murs musicaux	Haute	Terminé
Gestion des membres	Haute	Terminé
Gestion des sections	Haute	Terminé
Ajout d'albums et de morceaux	Haute	Terminé
Catalogue musical et recherche	Haute	Terminé
Profil utilisateur	Moyenne	Terminé
Interface responsive	Moyenne	Terminé
Tests et déploiement	Haute	Terminé
Système d'amis	Basse	Reporté en V2
Invitations avec acceptation	Basse	Reporté en V2
Favoris et statistiques	Basse	Reporté en V2

B - User Stories

Les besoins fonctionnels ont également été formulés sous forme de user stories afin de représenter les fonctionnalités du point de vue de l'utilisateur :

1. En tant qu'utilisateur non authentifié, je veux créer un compte afin d'accéder aux fonctionnalités de Music Wall.
2. En tant qu'utilisateur, je veux me connecter afin d'accéder à mes murs musicaux et à mon profil.
3. En tant qu'utilisateur authentifié, je veux créer un mur musical afin d'organiser mes découvertes musicales.
4. En tant que propriétaire d'un mur, je veux ajouter un utilisateur enregistré comme membre afin de partager le mur avec lui.
5. En tant que membre d'un mur, je veux créer et modifier des sections afin d'organiser les contenus musicaux du mur.
6. En tant que membre d'un mur, je veux ajouter un album ou un morceau depuis le catalogue afin de l'intégrer dans une section.

7. En tant que membre d'un mur, je veux modifier le statut d'écoute d'un morceau afin de savoir s'il reste à écouter ou s'il a déjà été écouté.
8. En tant que propriétaire d'un mur, je veux modifier son apparence afin de le personnaliser.

V - Conception de l'interface graphique

L'interface de Music Wall a été conçue autour d'une identité visuelle liée à l'univers musical et à l'idée de collection. L'objectif était de proposer une application identifiable visuellement, avec une identité chaleureuse et créative, tout en conservant une navigation claire et moderne entre les principales fonctionnalités.

La conception de l'interface a évolué progressivement avec le développement de l'application. Une première organisation des écrans a permis de définir les grandes zones fonctionnelles, puis les composants ont été affinés au fur et à mesure de l'intégration des fonctionnalités et des tests sur différentes tailles d'écran. J'ai utilisé Figma pour concevoir toutes mes maquettes.

A - Zoning

Le zoning permet de représenter les principales zones d'une interface sans détailler encore son apparence graphique. Il sert à déterminer la hiérarchie de l'information et la place des différents éléments avant de travailler sur leur présentation visuelle.

Pour Music Wall, les écrans principaux ont été organisés autour d'une structure commune comprenant :

• une zone de navigation ;
• une indication de la page actuellement consultée ;
• une zone de contenu principal ;
• des actions liées au contexte de la page.

La page d'un mur musical constitue l'écran le plus représentatif du projet. Elle est organisée en plusieurs zones principales :

• identification du mur ;
• actions de personnalisation et de gestion ;
• liste des membres ;
• sections musicales ;
• éléments musicaux contenus dans chaque section ;
• actions permettant d'ajouter une section ou du contenu musical.

Cette organisation permet à l'utilisateur de comprendre rapidement la structure d'un mur et d'identifier les différentes actions disponibles.

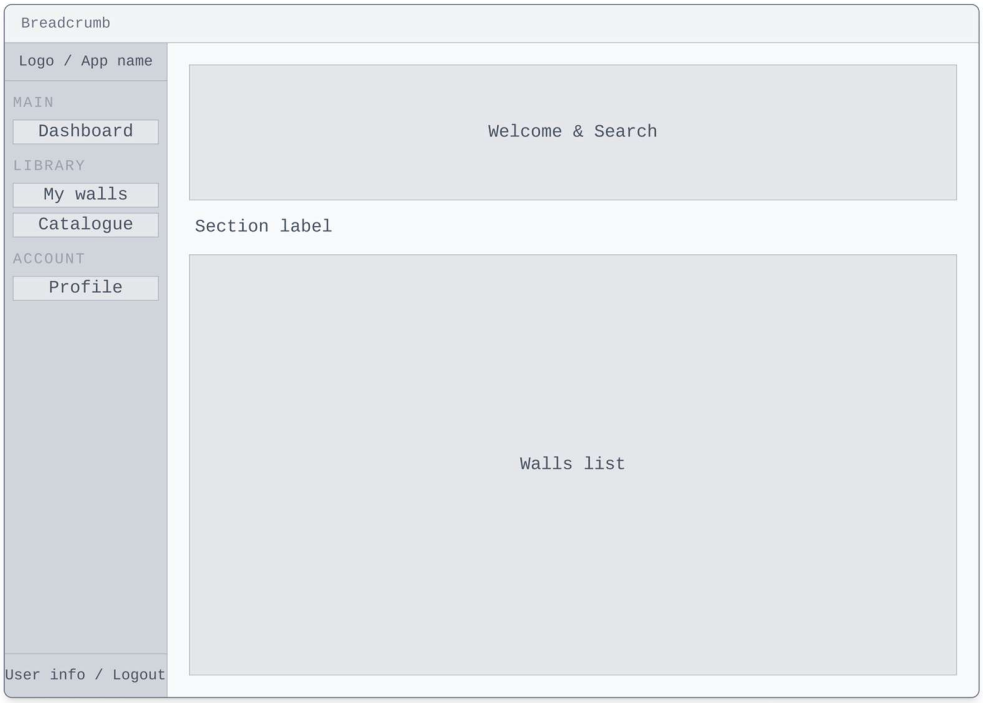


Figure 1 – Zoning page Dashboard (desktop)

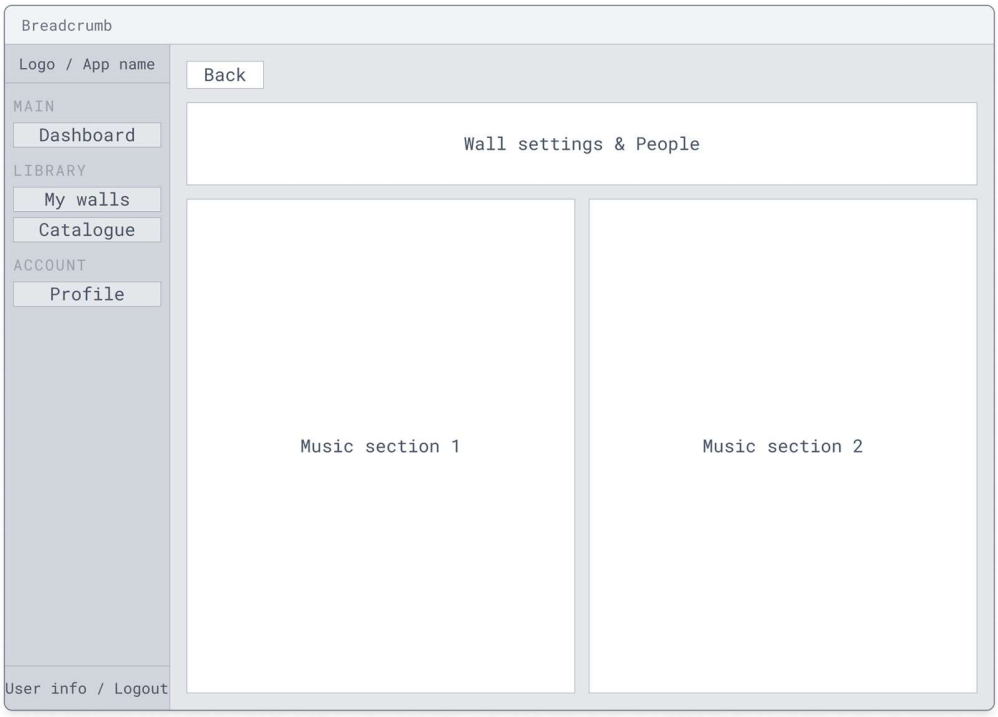


Figure 2 – Zoning page Wall Detail (desktop)

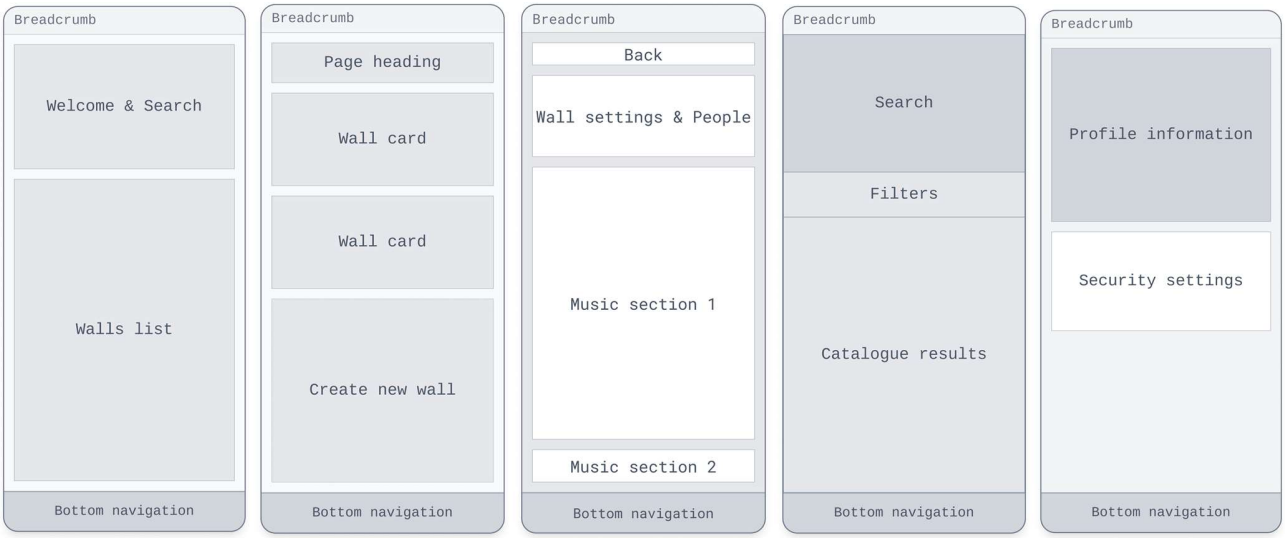


Figure 3 – Zoning mobile, pages : dashboard, my walls, wall detail, catalogue, profile

B - Wireframe

Le wireframe représente plus précisément la disposition des éléments d'une page, sans appliquer encore l'ensemble des couleurs, images et détails graphiques.

Pour Music Wall, le wireframe permet notamment de représenter la relation entre les différentes parties d'un mur musical. Le nom du mur et ses membres apparaissent en premier, suivis des différentes sections. Chaque section contient ensuite une liste de morceaux ou d'albums ainsi que les actions associées.

La conception de cette structure a été particulièrement importante car la notion de « mur » constitue le cœur de l'application. L'objectif était d'éviter une présentation similaire à une simple liste de lecture et de donner à chaque section une identité visuelle identifiable.

Les autres pages de l'application conservent une organisation plus classique :

• Dashboard : introduction, recherche musicale et accès rapide aux murs ;
• My walls : liste des murs accessibles et création d'un nouveau mur ;
• Catalogue : recherche, filtres et affichage des artistes, albums et morceaux ;
• Profile : informations personnelles et paramètres de sécurité.

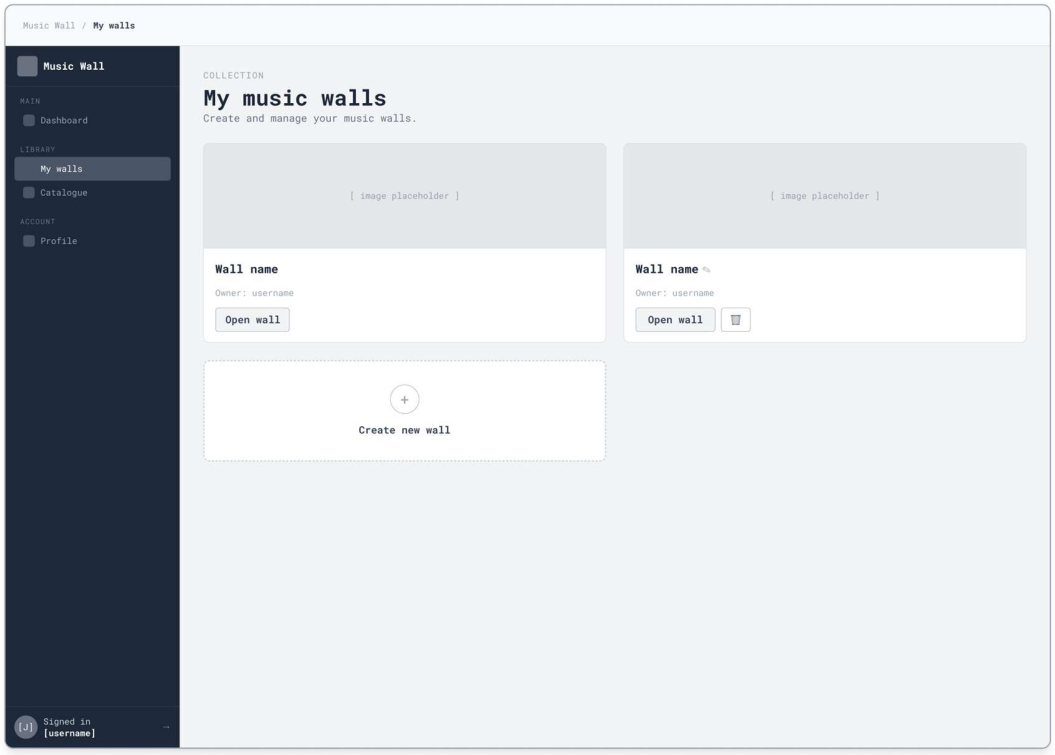


Figure 4 – Wireframe desktop, page My walls

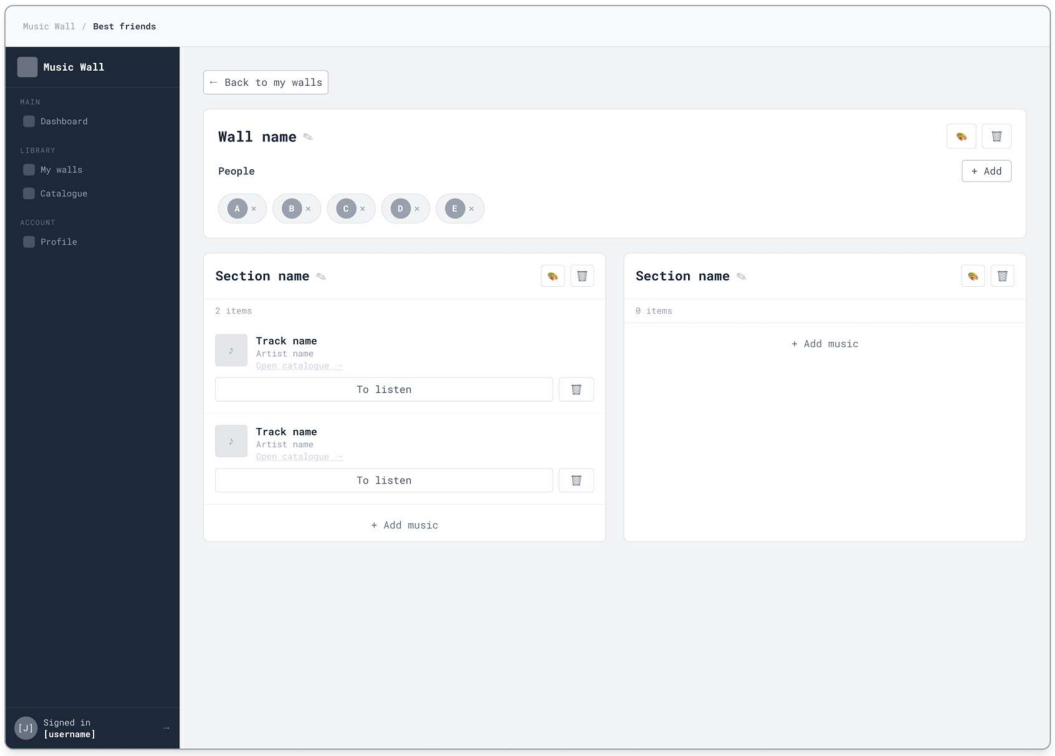


Figure 5 – Wireframe desktop, page Wall detail

C - Charte graphique

La charte graphique de Music Wall repose sur une palette claire associant des tons bleu-vert à une couleur corail utilisée comme accent principal. Le corail met en valeur les actions et les éléments interactifs, tandis que le bleu

pétrole structure les titres et les textes importants. Les fonds très clairs et les surfaces blanches permettent de conserver une interface lisible et légère.

Des couleurs secondaires sont utilisées pour les bordures, les séparateurs, les textes secondaires et les messages d'erreur. Les sections musicales disposent également de plusieurs teintes décoratives afin de permettre une personnalisation visuelle des murs tout en restant cohérentes avec l'identité générale de l'application.

La typographie repose sur deux familles complémentaires. DM Serif Display est utilisée principalement pour les titres et les éléments à caractère éditorial, tandis que Nunito est employée pour les textes courants, la navigation, les boutons, les formulaires et les métadonnées. Les polices sont intégrées localement dans le projet avec @fontsource.

Le logo reprend directement le concept de Music Wall. Les blocs du symbole évoquent à la fois un mur et les barres d'un égaliseur, tandis que la forme circulaire rappelle un disque et des ondes musicales. Le logo blanc est utilisé sur la barre latérale turquoise afin de garantir un contraste clair.



Figure 6 – Logo blanc sur le fond turquoise

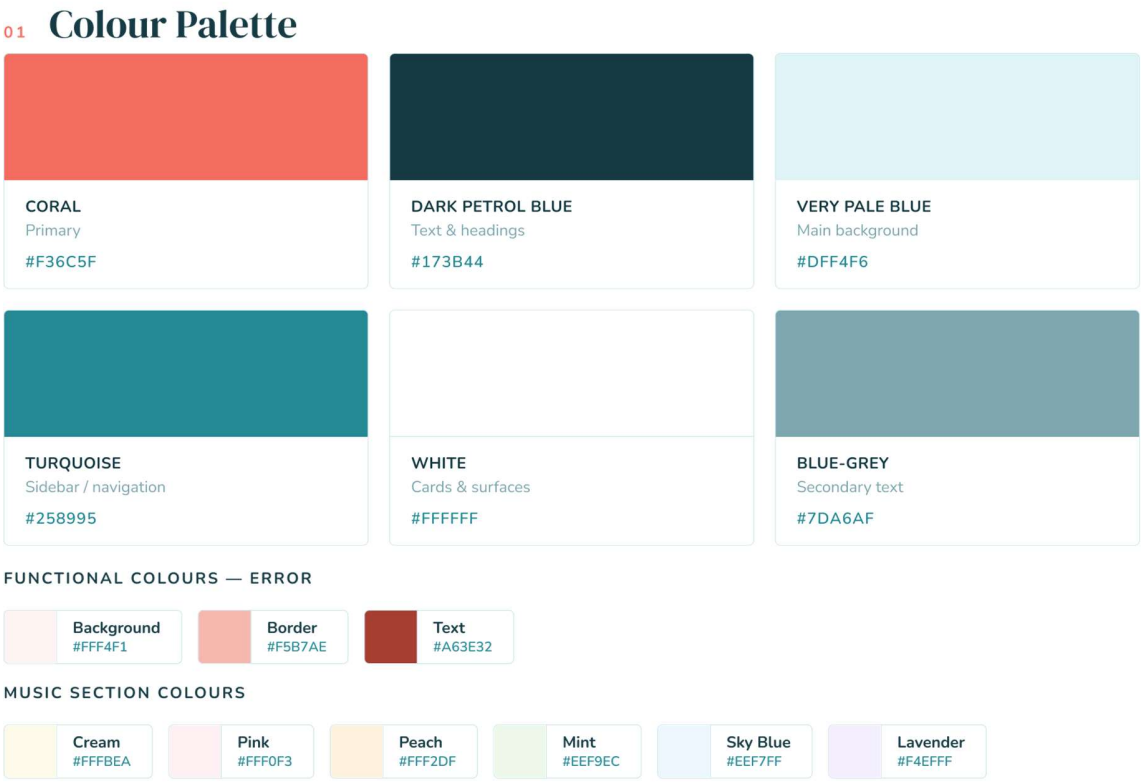


Figure 7 - Charte graphique

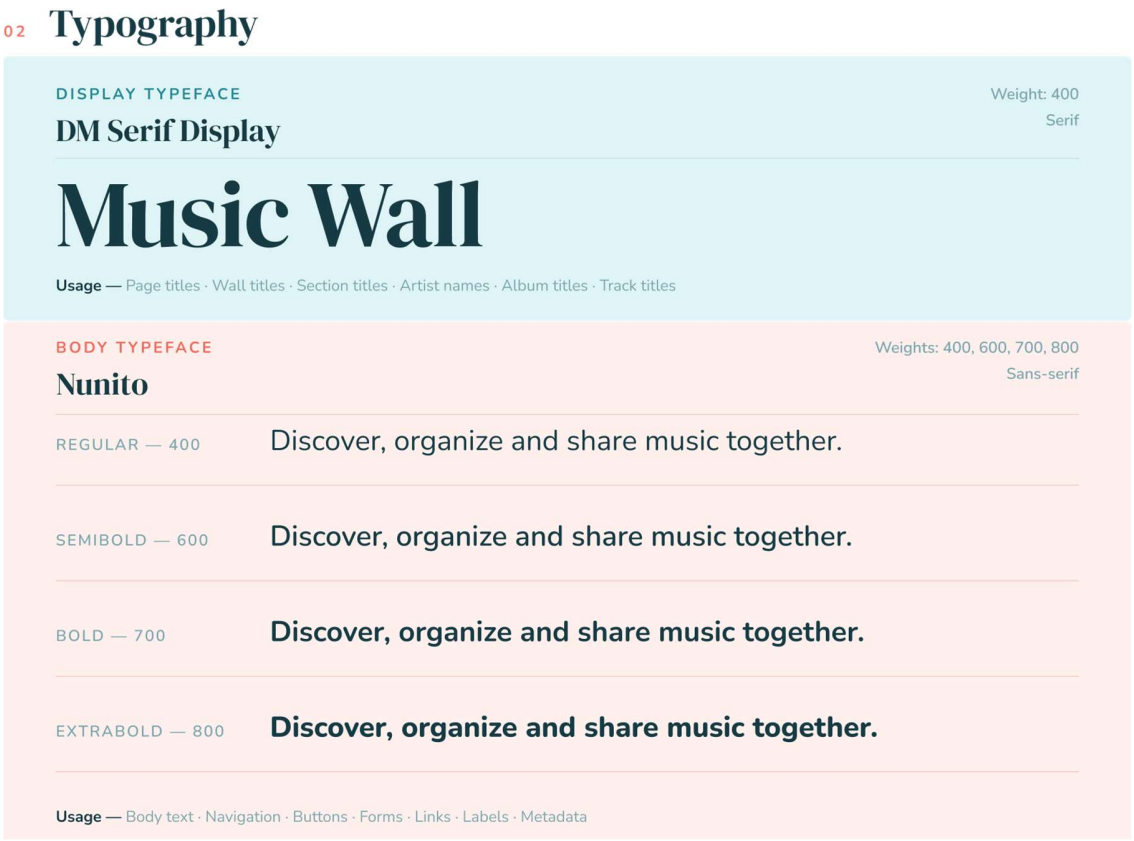


Figure 8 - Typographie

D - Maquettage

Le maquettage correspond à une représentation plus détaillée de l'interface intégrant les couleurs, les typographies, les boutons, les cartes et les autres éléments graphiques.

L'identité visuelle de Music Wall s'appuie notamment sur le principe de cartes. Les murs, les sections, les contenus musicaux et les zones de profil sont représentés sous forme de surfaces clairement séparées du fond de la page.

Les murs peuvent être personnalisés avec différentes couleurs et différents fonds graphiques. Cette possibilité permet à chaque espace partagé d'avoir une identité visuelle différente tout en conservant la structure générale de l'application.

Une attention particulière a été portée à la page de détail d'un mur. Les sections utilisent une présentation distincte du fond du mur et regroupent les morceaux ou albums qui leur appartiennent. Les informations principales restent ainsi lisibles même lorsque l'utilisateur choisit un fond relativement chargé.

Le catalogue possède une identité visuelle propre avec une zone de recherche mise en avant, suivie de filtres permettant d'afficher tous les résultats ou uniquement les artistes, albums ou morceaux.

La page profil présente l'avatar, le nom d'utilisateur et la biographie dans une zone visuelle principale, tandis que les fonctions liées à la sécurité, comme le changement de mot de passe, sont regroupées dans une section distincte.

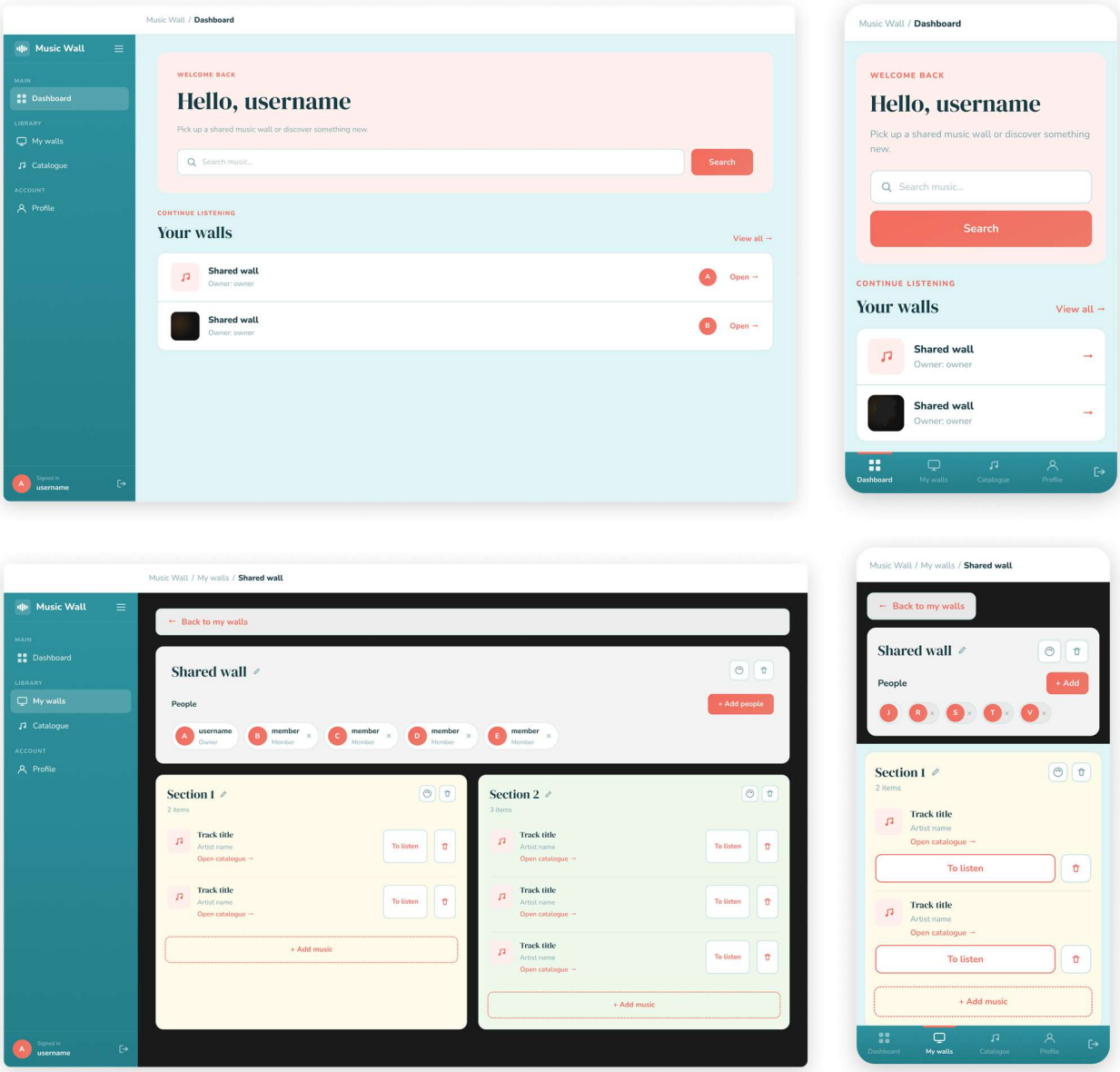


Figure 9 - Maquettes desktop/mobile

E - Responsive design

Music Wall a été conçu pour être utilisable aussi bien sur ordinateur que sur smartphone.

Le projet n'a pas suivi une approche strictement « mobile first ». Le concept principal de l'application, basé sur un mur musical composé de sections et de différents contenus, étant particulièrement lisible sur un écran large, j'ai commencé par définir l'organisation de l'interface desktop.

Les composants ont ensuite été adaptés progressivement aux écrans plus petits à l'aide de règles CSS responsive et de media queries.

Sur desktop, la navigation principale peut exploiter davantage l'espace disponible et les informations peuvent être présentées horizontalement.

Sur smartphone, plusieurs adaptations ont été réalisées :

• remplacement de la navigation desktop par une navigation fixe en bas de l'écran ;
• réduction des espaces et de certains éléments graphiques ;
• adaptation de la taille des titres ;
• réorganisation des boutons lorsque plusieurs actions ne peuvent plus tenir sur une même ligne ;
• retour automatique à la ligne des noms de murs et des titres musicaux longs ;
• adaptation de la disposition des membres d'un mur ;
• réorganisation des cartes de morceaux afin de laisser suffisamment de place au titre et aux informations musicales ;
• présentation des filtres du catalogue sous une forme adaptée aux petits écrans ;
• adaptation des zones de recherche et des formulaires ;
• conservation de zones tactiles suffisamment grandes pour les principales actions.

L'interface a été vérifiée sur plusieurs largeurs représentatives de smartphones, notamment autour de 360, 390 et 430 pixels, ainsi que sur des formats plus larges.

L'objectif n'était pas de reproduire exactement la disposition desktop sur un écran plus petit, mais de conserver les mêmes fonctionnalités tout en adaptant leur présentation aux contraintes du smartphone.

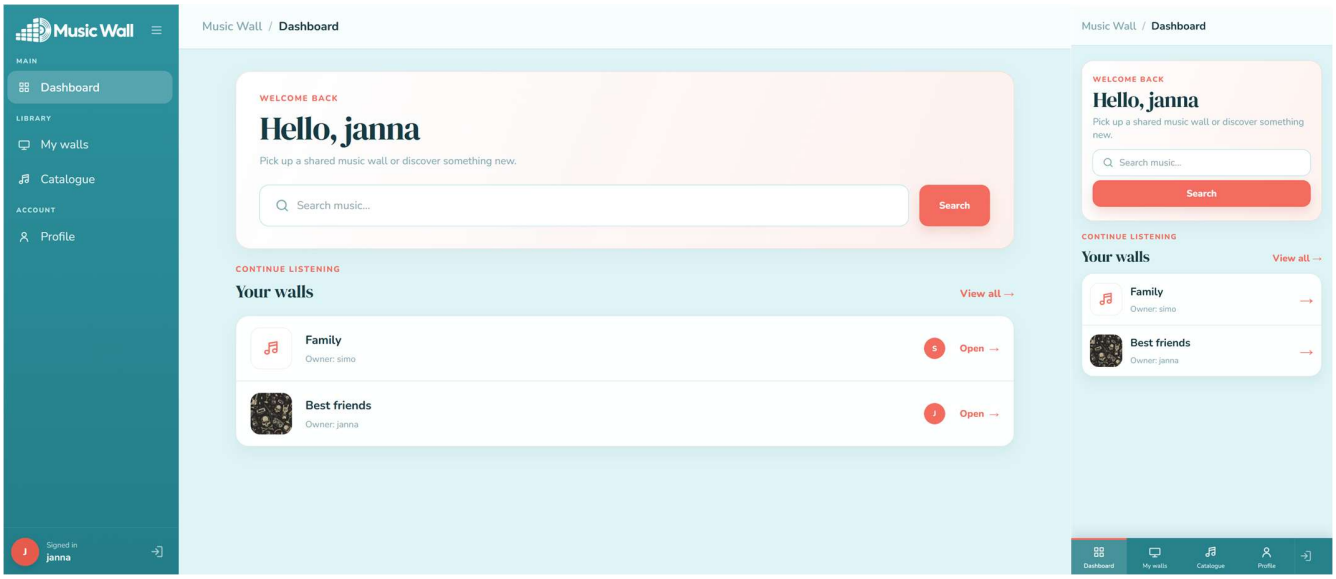


Figure 10 – Page Dashboard desktop / mobile

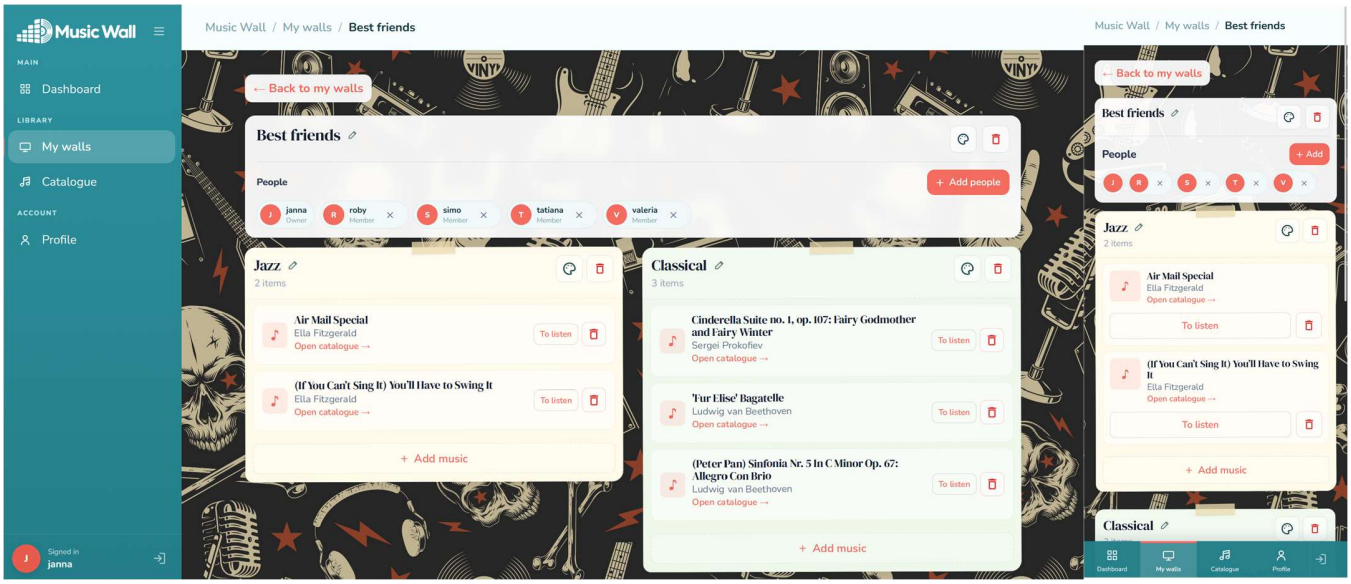


Figure 11 - Page Wall detail desktop / mobile

VI - Conception de la base de données

La base de données de Music Wall stocke les comptes utilisateurs, les murs musicaux, leurs sections et leurs contenus, ainsi que le catalogue local composé d'artistes, d'albums, de morceaux et de genres.

Pour structurer ces données, j'ai utilisé la méthode Merise en distinguant trois niveaux de modélisation : le Modèle Conceptuel de Données, le Modèle Logique de Données et le Modèle Physique de Données. Cette progression m'a permis de partir des besoins métier, de définir ensuite les relations et les clés nécessaires, puis de représenter leur implémentation concrète dans PostgreSQL.

A - Modèle Conceptuel de Données – MCD

Le Modèle Conceptuel de Données représente les principales données métier de Music Wall ainsi que les associations et les cardinalités qui les relient. À ce niveau, le modèle reste indépendant de la technologie utilisée : il ne contient ni types PostgreSQL ni clés étrangères.

Le modèle distingue principalement les utilisateurs, les murs musicaux, les sections et les éléments ajoutés aux murs, ainsi que les artistes, albums, morceaux et genres constituant le catalogue musical.

Un utilisateur peut posséder plusieurs murs et participer à plusieurs murs partagés. Un mur peut contenir plusieurs sections et chaque section peut contenir plusieurs éléments musicaux. Le propriétaire d'un mur n'est pas ajouté une seconde fois dans la liste de ses membres.

Dans le catalogue, un artiste peut être associé à plusieurs albums et morceaux. Un morceau peut éventuellement appartenir à un album. Les albums et les morceaux peuvent être associés à plusieurs genres.

Un élément musical ajouté dans une section référence soit un morceau, soit un album du catalogue. Une contrainte métier XOR impose qu'une et une seule de ces deux associations soit renseignée pour un même élément.

Les noms des entités ont été conservés en anglais afin de rester cohérents avec la terminologie utilisée dans le code de l'application.

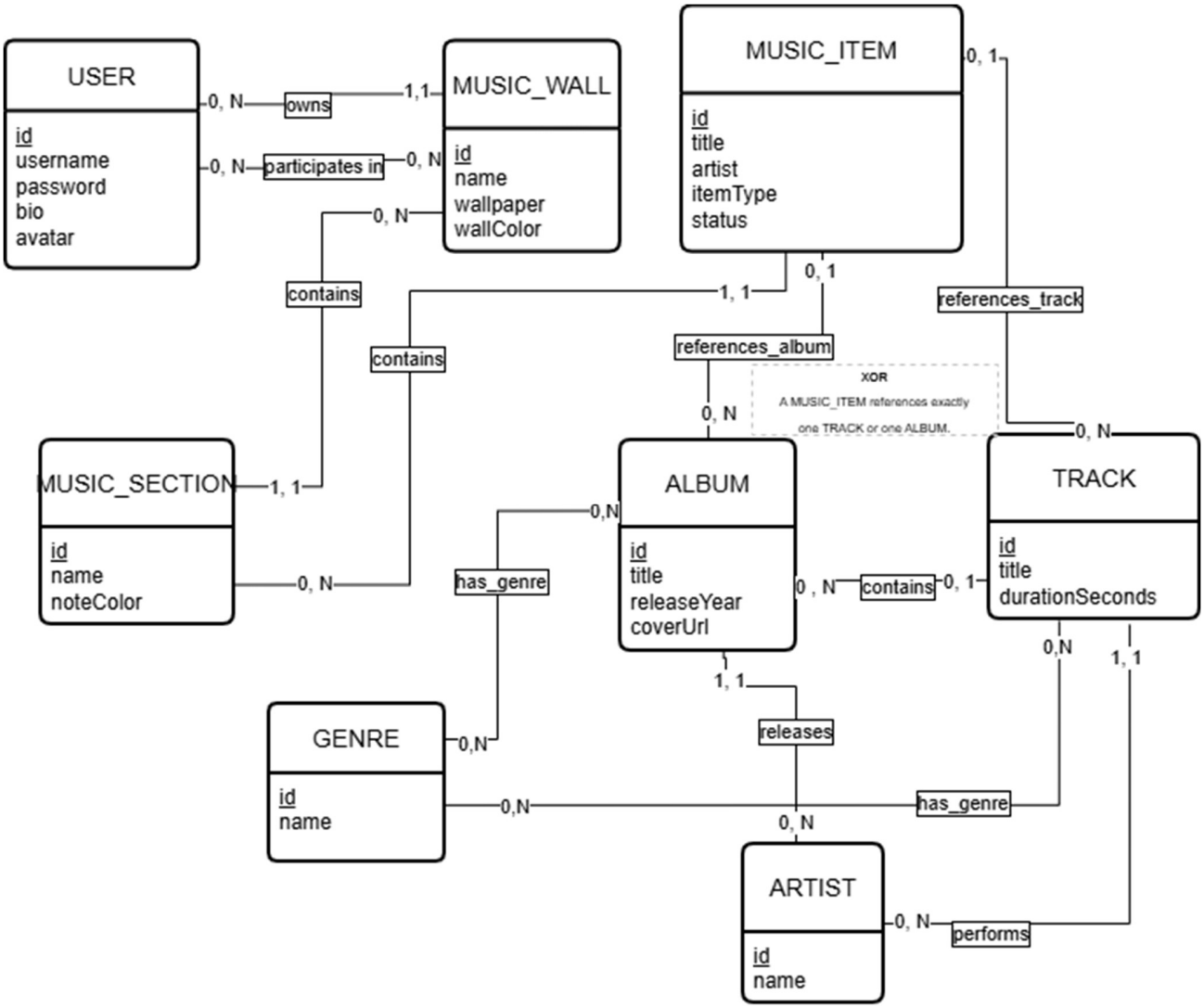


Figure 12 - Diagramme MCD

B - Modèle Logique de Données – MLD

Le Modèle Logique de Données traduit le MCD sous une forme relationnelle. Les entités deviennent des tables disposant de clés primaires, tandis que les associations sont traduites par des clés étrangères ou par des tables d'association.

Les relations plusieurs-à-plusieurs du MCD conduisent notamment à la création de WALL_MEMBERS, ALBUM_GENRE et TRACK_GENRE. Les relations un-à-plusieurs sont représentées par des clés étrangères placées du côté plusieurs. Par exemple, MUSIC_WALL contient owner_id, MUSIC_SECTION contient wall_id et MUSIC_ITEM contient section_id.

La relation facultative entre TRACK et ALBUM est représentée par album_id.

Enfin, MUSIC_ITEM possède deux références possibles vers le catalogue, catalog_track_id et catalog_album_id. Une règle métier impose qu'une et une seule de ces deux références soit renseignée pour un même élément musical.

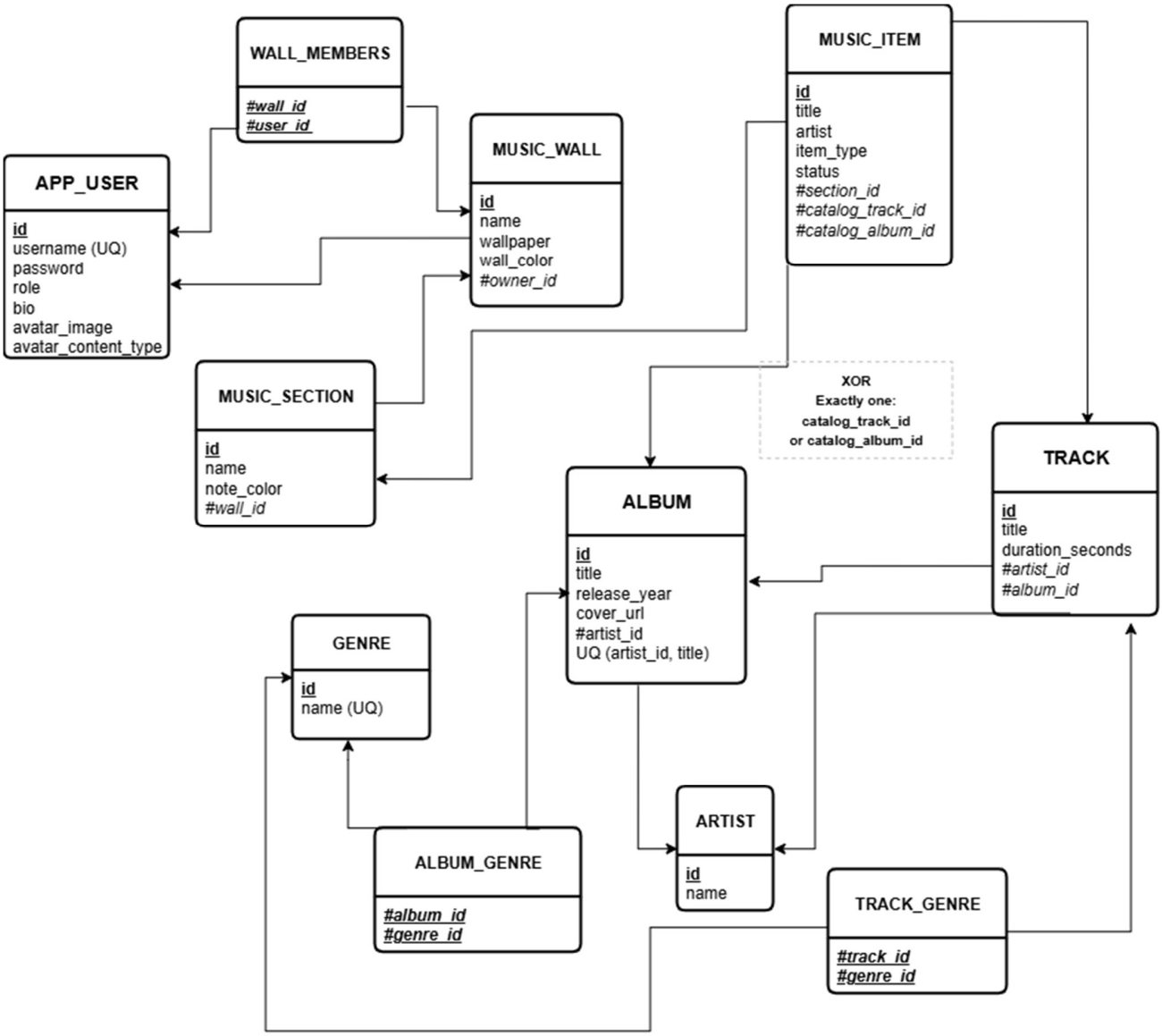


Figure 13 - Diagramme MLD

C - Modèle Physique de Données - MPD

Le Modèle Physique de Données représente l'implémentation concrète du modèle relationnel dans PostgreSQL. Il précise les noms réels des tables et des colonnes, leurs types, les clés primaires et étrangères, les contraintes d'unicité ainsi que la nullabilité des données.

Les identifiants principaux sont stockés sous forme de BIGINT générés automatiquement. Les données textuelles utilisent principalement des colonnes VARCHAR de différentes longueurs. L'avatar utilisateur est stocké dans une colonne BYTEA.

Les relations sont garanties par des contraintes de clés étrangères. Les relations plusieurs-à-plusieurs sont matérialisées par les tables WALL_MEMBERS, ALBUM_GENRE et TRACK_GENRE. Chacune de ces tables utilise une clé primaire composée des deux clés étrangères qu'elle contient.

Plusieurs contraintes d'unicité complètent le modèle. Le nom d'utilisateur, le nom d'un artiste et le nom d'un genre sont uniques. Pour les albums, la combinaison de artist_id et title doit également être unique.

Les valeurs des énumérations Java sont persistées sous forme de chaînes de caractères. PostgreSQL applique également des contraintes CHECK sur certaines de ces colonnes, notamment item_type, status et wallpaper, afin de limiter les valeurs possibles.

Dans le projet, la structure principale des tables est créée et mise à jour à partir des entités JPA avec Hibernate. Le fichier schema.sql complète cette structure avec l'extension pg_trgm et des index GIN utilisés pour améliorer les recherches textuelles dans le catalogue.



Figure 14 - Diagramme MPD

VII - Conception de l'application

A - Use Case

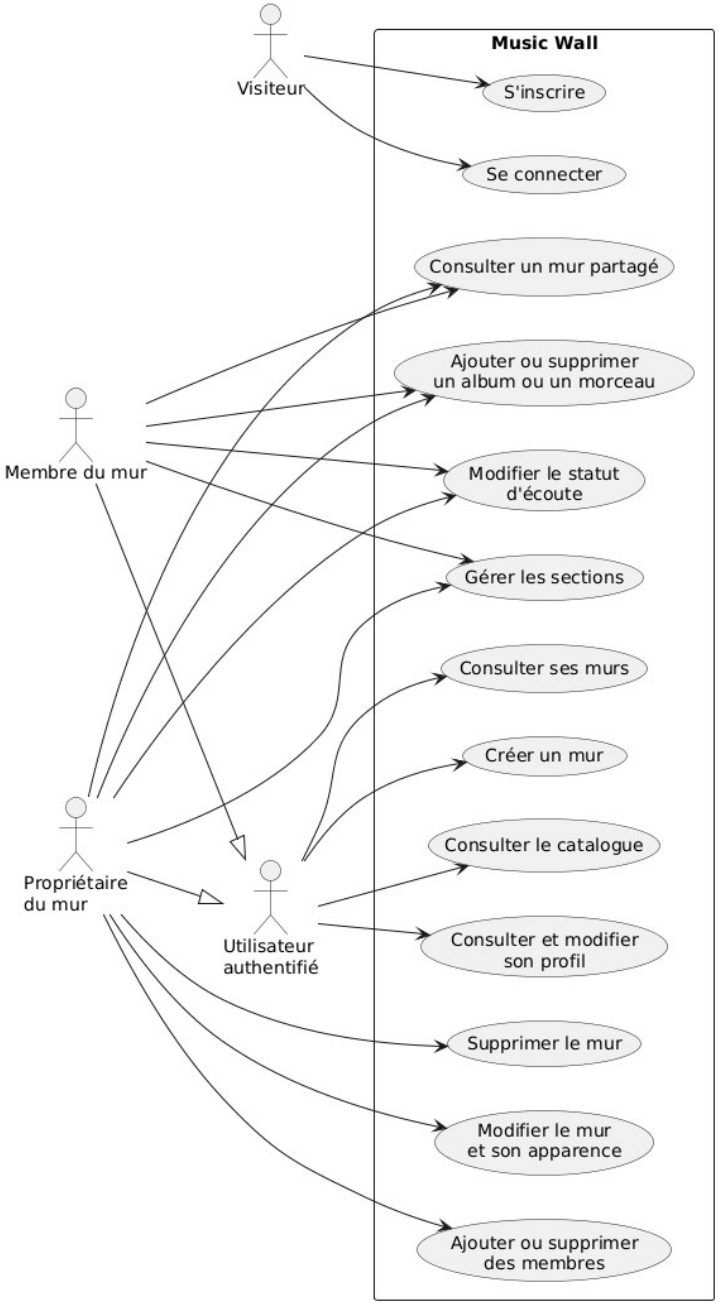


Figure 15 - Diagramme de cas d'utilisation de Music Wall

Ce diagramme présente les principales fonctionnalités accessibles selon le rôle de l'utilisateur. Un utilisateur authentifié peut gérer son profil et ses murs. Un membre peut gérer le contenu d'un mur partagé, tandis que certaines opérations, notamment la modification du mur et la gestion des membres, sont réservées à son propriétaire.

B - Diagramme de classes

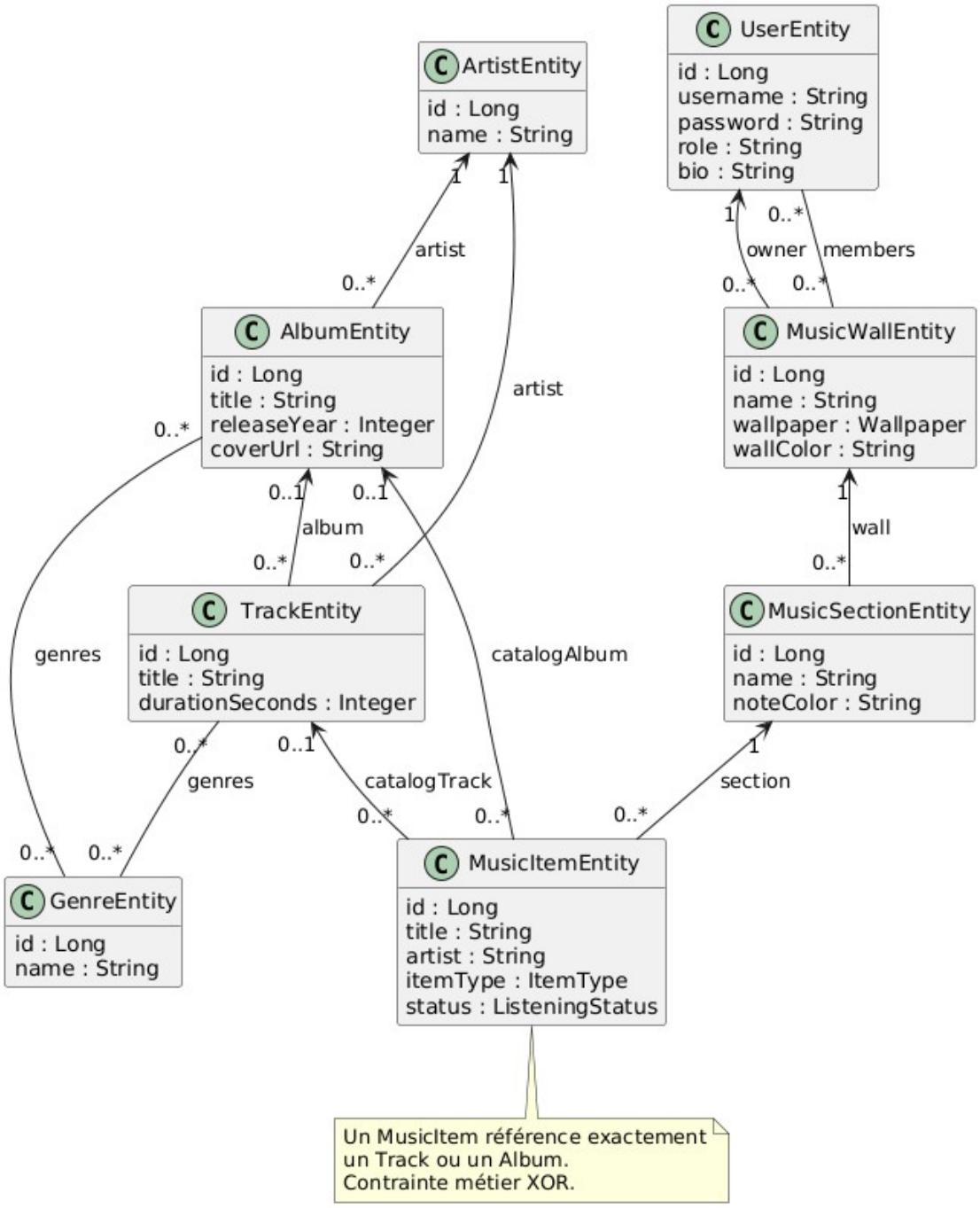


Figure 16 - Diagramme de classes principales de Music Wall

Le diagramme de classes représente les principales entités Java utilisées dans l'application et leurs relations. Il montre notamment les relations entre les utilisateurs, les murs, les sections et les éléments musicaux, ainsi que la structure du catalogue local.

C - Diagramme de séquence

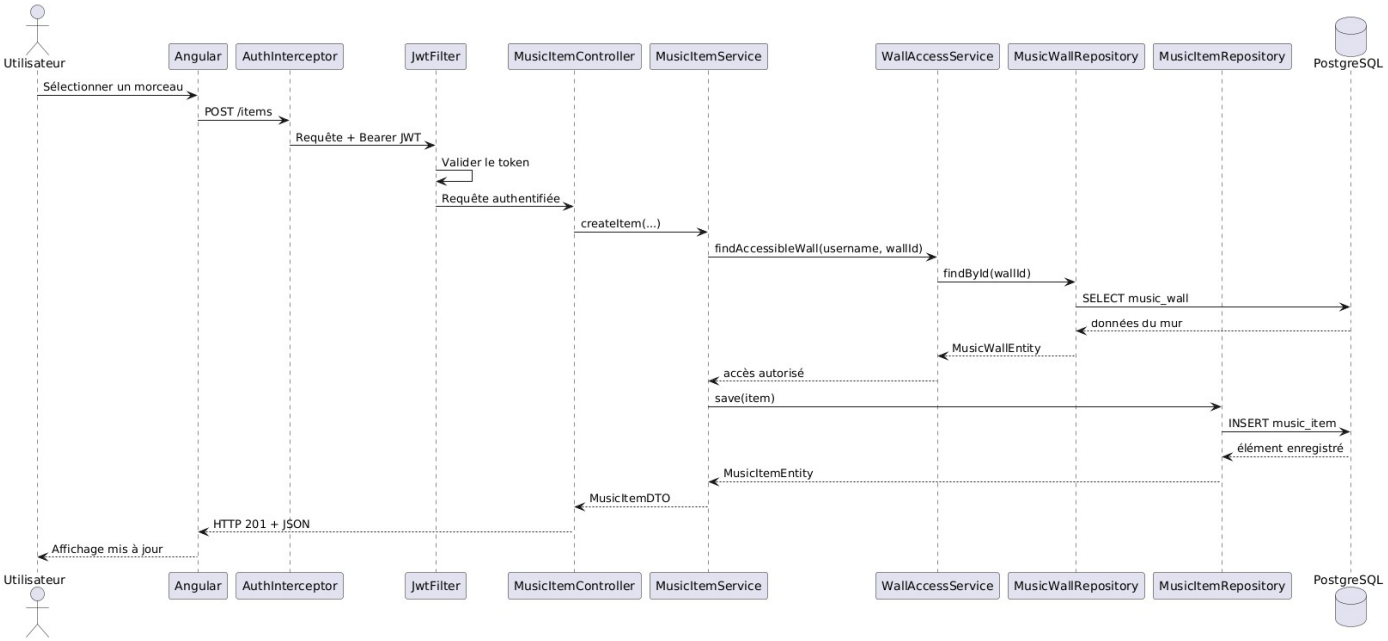


Figure 17 - Diagramme de séquence de l'ajout d'un morceau à une section

Ce diagramme représente le déroulement d'une requête permettant d'ajouter un morceau à une section. Le token JWT est ajouté à la requête puis vérifié par le backend. Le service contrôle ensuite les droits d'accès au mur avant d'enregistrer l'élément dans PostgreSQL.

VIII. Architecture logicielle

Music Wall repose sur une architecture client-serveur séparant le frontend, le backend et la base de données.

Le frontend est développé avec Angular et fonctionne dans le navigateur sous la forme d'une Single Page Application. Il est responsable de l'affichage de l'interface, de la navigation, de la gestion des formulaires et des interactions avec l'utilisateur.

Le frontend communique avec le backend Spring Boot à travers une API REST. Les échanges sont réalisés au format JSON à l'aide de requêtes HTTP utilisant principalement les méthodes GET, POST, PUT et DELETE.

Pour les requêtes nécessitant une authentification, le token JWT stocké côté client est ajouté automatiquement dans l'en-tête Authorization par un intercepteur Angular.

Le backend Spring Boot reçoit les requêtes de l'application Angular et assure le traitement des données et de la logique métier. Il est organisé en plusieurs couches afin de séparer les responsabilités :

- les Controllers exposent les endpoints REST et reçoivent les requêtes HTTP ;

- les Services contiennent la logique métier de l'application ;

- les Repositories assurent l'accès aux données avec Spring Data JPA ;

- les Entities représentent les données persistées dans PostgreSQL ;

- les DTO définissent les données échangées entre l'API et le frontend.

Spring Security intervient également dans le traitement des requêtes protégées. Le JwtFilter extrait et valide le token JWT puis charge l'utilisateur authentifié dans le SecurityContext.

Les autorisations liées aux murs musicaux sont ensuite vérifiées côté backend. Le WallAccessService permet notamment de contrôler si l'utilisateur est propriétaire d'un mur, membre de celui-ci ou extérieur au mur.

La persistance est assurée par PostgreSQL. Spring Data JPA et Hibernate permettent de faire le lien entre les entités Java et les tables de la base de données.

Le fonctionnement général de l'application peut ainsi être résumé de la manière suivante :

Utilisateur -> Interface Angular -> Service Angular-> Requête HTTP REST-> Spring Security / JwtFilter-> Controller-> Service-> Repository-> PostgreSQL

La réponse suit ensuite le chemin inverse jusqu'au frontend, où les données JSON reçues sont utilisées pour mettre à jour l'interface.

Cette séparation entre le frontend, le backend et la base de données facilite l'organisation du projet, la maintenance du code et l'évolution de l'application. Chaque partie possède une responsabilité clairement définie et peut être modifiée sans devoir restructurer l'ensemble du système.

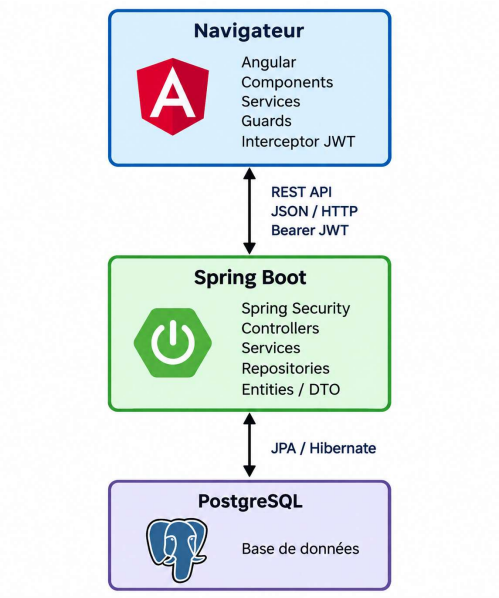
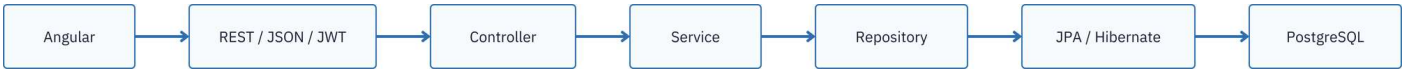


Figure 18 - Architecture logicielle de Music Wall

IX. Architecture multicouche

Music Wall utilise une architecture multicouche afin de séparer les différentes responsabilités de l'application.

Cette organisation permet de distinguer la présentation, la logique métier et l'accès aux données. Chaque couche possède un rôle précis et communique avec les autres à travers des interfaces clairement définies.



A - Couche de présentation

La couche de présentation correspond au frontend Angular.

Elle est responsable de l'affichage de l'interface et des interactions avec l'utilisateur. Elle comprend notamment les composants Angular, les formulaires, la navigation et les services utilisés pour communiquer avec l'API.

Les services Angular utilisent HttpClient pour envoyer des requêtes HTTP au backend Spring Boot.

L'application utilise également un guard pour protéger certaines routes et un interceptor pour ajouter automatiquement le token JWT dans l'en-tête Authorization des requêtes authentifiées.

La couche de présentation ne communique pas directement avec la base de données. Toutes les opérations passent par l'API REST.

B - Couche de traitement et logique métier

Le backend est développé avec Spring Boot.

Les Controllers constituent le point d'entrée de l'API. Ils reçoivent les requêtes HTTP, récupèrent les données transmises par le frontend et appellent les services correspondants.

Les Services contiennent la logique métier de l'application. Ils appliquent les règles nécessaires avant l'accès aux données.

Dans Music Wall, cette couche gère notamment :

- la création et la modification des murs musicaux ;

- la gestion des sections et des éléments musicaux ;

- l'ajout et la suppression de membres ;

- la vérification des droits du propriétaire et des membres ;

- la gestion du profil utilisateur ;

- l'authentification et le changement de mot de passe ;

- la consultation et la recherche dans le catalogue musical.

Par exemple, lorsqu'un utilisateur souhaite modifier un mur, le service ne se contente pas d'effectuer une modification en base. Il vérifie également que l'utilisateur possède les droits nécessaires.

Le WallAccessService centralise une partie de ces contrôles d'autorisation en vérifiant si l'utilisateur est propriétaire ou membre du mur concerné.

C - Couche d'accès aux données

La couche d'accès aux données est basée sur Spring Data JPA.

Les interfaces Repository permettent d'effectuer les opérations de lecture et d'écriture sans manipuler directement les requêtes SQL pour les opérations courantes.

Hibernate assure le mapping entre les entités Java et les tables PostgreSQL.

Les entités représentent les principales données persistantes de l'application, notamment les utilisateurs, les murs, les sections, les éléments musicaux ainsi que les artistes, albums, morceaux et genres du catalogue.

PostgreSQL constitue la couche de stockage finale de l'application.

D - Communication entre les couches

Lorsqu'un utilisateur effectue une action depuis l'interface, le flux suit généralement les étapes suivantes :

Utilisateur-> Component Angular-> Service Angular-> Requête HTTP REST-> Spring Security / JwtFilter-> Controller-> Service-> Repository-> PostgreSQL

La réponse suit ensuite le chemin inverse jusqu'au frontend.

Par exemple, lorsqu'un utilisateur ajoute un morceau dans une section, Angular envoie une requête à l'API. Le backend authentifie l'utilisateur, le Controller transmet la demande au Service, le Service vérifie les droits d'accès et la règle métier, puis le Repository enregistre les données dans PostgreSQL.

Cette architecture permet de séparer clairement l'interface, la logique métier et la persistance. Elle facilite la compréhension du code, les tests, la maintenance et les évolutions futures de l'application.

X. Sécurité

La sécurité de Music Wall est prise en compte à plusieurs niveaux : authentification des utilisateurs, contrôle des autorisations, protection des mots de passe, validation des données entrantes et configuration des échanges entre le frontend et le backend.

L'application utilise principalement Spring Security, JWT et BCrypt côté backend. Angular participe également à la gestion de l'authentification en stockant le token reçu après la connexion et en l'ajoutant aux requêtes protégées.

A. Authentification / autorisation

L'authentification permet de vérifier l'identité d'un utilisateur, tandis que l'autorisation détermine les actions qu'il peut effectuer une fois authentifié.

Lors de la connexion, Music Wall utilise l'AuthenticationManager de Spring Security pour vérifier le nom d'utilisateur et le mot de passe. Si l'authentification réussit, un token JWT est généré et retourné au frontend.

Spring Security définit également les ressources publiques et protégées de l'API. Les endpoints d'authentification sont publics. Certaines opérations de lecture du catalogue et des profils sont également accessibles sans authentification, tandis que les autres requêtes nécessitent un utilisateur authentifié.

Les droits liés aux murs musicaux sont vérifiés séparément dans la logique métier. Le WallAccessService distingue notamment le propriétaire d'un mur, ses membres et les utilisateurs extérieurs. Certaines opérations, comme la modification du mur ou la gestion de ses membres, sont réservées au propriétaire.

```java
public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                    request.getUsername(),
                    request.getPassword()
            )
    );

    UserEntity user = userRepository.findByUsername(request.getUsername())
            .orElseThrow(() -> new BusinessException(message: "User not found"));

    return createResponse(user);
}
```

Figure 19 - Exemple : authentification - AuthService.java

AuthenticationManager vérifie les identifiants. Si l'authentification réussit, on récupère l'utilisateur puis on génère la réponse contenant le JWT.

```java
public MusicWallEntity findOwnedWall(String username, Long wallId) {
    MusicWallEntity wall = findWall(wallId);
    if (!isOwner(wall, username)) {
        throw new ForbiddenException(
            message: "Only the wall owner can perform this action");
    }
    return wall;
}
```

Figure 20 - Exemple : autorisation métier - WallAccessService.java

Le fait d'être connecté ne donne pas automatiquement tous les droits.

B. XSS

Une attaque XSS (Cross-Site Scripting) consiste à injecter du contenu JavaScript malveillant dans une page afin qu'il soit exécuté dans le navigateur d'un autre utilisateur.

Dans Music Wall, les données textuelles provenant de l'application sont principalement affichées avec l'interpolation Angular. Angular échappe automatiquement les valeurs insérées de cette manière dans le HTML, ce qui réduit le risque qu'un texte fourni par un utilisateur soit interprété comme du code exécutable.

Cette protection est notamment importante pour les contenus qui peuvent être saisis ou affichés dynamiquement, comme le nom d'un mur, le titre d'un morceau ou le nom d'un artiste.

```html
<div>
  <h3>{{ item.title }}</h3>
  <p>{{ item.artist }}</p>
</div>
```

Figure 21 – Exemple : Protection contre les attaques XSS - music-item.component.html

Angular ne place pas directement cette valeur comme du HTML exécutable. Avec l'interpolation, le contenu est échappé avant l'affichage.

C. Injection SQL

Une injection SQL consiste à introduire du code SQL malveillant dans une donnée envoyée à l'application afin de modifier le comportement d'une requête.

Music Wall utilise Spring Data JPA pour la majorité des accès aux données. Les données fournies par l'utilisateur sont transmises sous forme de paramètres et ne sont pas concaténées directement dans une chaîne SQL.

Certaines recherches du catalogue utilisent des requêtes natives PostgreSQL. Même dans ce cas, les valeurs saisies sont passées à la requête avec des paramètres nommés, par exemple : query. Cette approche permet de séparer la structure de la requête SQL de la valeur fournie par l'utilisateur.

```
@Query(value = """
        SELECT a.*
        FROM artist a
        WHERE lower(a.name) LIKE '%' || lower(:query) || '%'
           OR lower(a.name) % lower(:query)
        ORDER BY
            CASE
                WHEN lower(a.name) = lower(:query) THEN 3
                WHEN lower(a.name) LIKE lower(:query) || '%' THEN 2
                WHEN lower(a.name) LIKE '%' || lower(:query) || '%' THEN 1
                ELSE 0
            END DESC,
            similarity(lower(a.name), lower(:query)) DESC,
            a.name ASC
        """, nativeQuery = true)
List<ArtistEntity> searchSimilar(@Param("query") String query);
```

Figure 22 - Exemple : Protection contre les injections SQL - ArtistRepository.java

Même s'il s'agit d'une requête native, la saisie utilisateur n'est pas injectée directement dans la chaîne SQL : Spring se charge de lier la valeur au paramètre :query.

D. CSRF

Une attaque CSRF (Cross-Site Request Forgery) cherche à faire envoyer par le navigateur d'un utilisateur authentifié une requête qu'il n'a pas volontairement déclenchée.

Dans Music Wall, l'authentification est stateless et repose sur un token JWT transmis explicitement dans l'en-tête Authorization. Le token n'est pas utilisé comme cookie d'authentification automatiquement envoyé par le navigateur.

Pour cette raison, la protection CSRF intégrée de Spring Security est désactivée dans la configuration actuelle. Cette architecture réduit l'exposition au scénario classique de CSRF basé sur les cookies.

Les origines autorisées sont également limitées par la configuration CORS.

```
.csrf(csrf -> csrf.disable())
.cors(cors -> cors.configurationSource(corsConfigurationSource()))
.sessionManagement(session -> session.sessionCreationPolicy(
        SessionCreationPolicy.STATELESS
))
```

Figure 23 - Protection contre les attaques CSRF - SecurityConfig.java

Le risque d'attaque CSRF survient principalement lorsque le navigateur transmet automatiquement les identifiants, notamment via les cookies. Dans ce cas précis, le token JWT est envoyé explicitement dans l'en-tête Authorization.

E. JWT

Music Wall utilise JWT (JSON Web Token) afin de conserver une authentification sans session serveur.

Après une inscription ou une connexion réussie, le backend génère un token contenant le nom d'utilisateur comme sujet. Le token possède également une date de création et une date d'expiration et il est signé avec une clé secrète.

Le frontend conserve ensuite le token et un interceptor Angular l'ajoute automatiquement aux requêtes HTTP sous la forme :

Authorization: Bearer <token>

Côté backend, le JwtFilter extrait le token, vérifie sa validité et charge l'utilisateur authentifié dans le SecurityContext de Spring Security. Les Controllers peuvent ensuite récupérer l'identité de l'utilisateur authentifié.

```
public String generateToken(String username) {
    return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(getSigningKey())
            .compact();
}
```

Figure 24 - Exemple : JSON Web Token - JwtUtil.java

```
if (token) {
  request = request.clone({
    setHeaders: {
      Authorization: 'Bearer ' + token
    }
  });
}
```

Figure 25 - Exemple : Intercepteur JWT - auth.interceptor.ts

F. BCrypt

Les mots de passe des utilisateurs ne sont jamais enregistrés en clair dans la base de données.

Music Wall utilise BCrypt à travers le PasswordEncoder de Spring Security. Lors de l'inscription, le mot de passe fourni par l'utilisateur est transformé en hash avant l'enregistrement du compte.

Lors de la connexion, Spring Security compare le mot de passe fourni avec le hash enregistré. Il n'est donc pas nécessaire de récupérer ou de connaître le mot de passe original.

```
UserEntity user = new UserEntity();
user.setUsername(request.getUsername());
user.setPassword(passwordEncoder.encode(request.getPassword()));
user.setRole(role: "USER");
userRepository.save(user);
```

Figure 26 - Exemple : encryption BCrypt - AuthService.java

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Figure 27 - Exemple : encryption BCrypt - SecurityConfig.java

Le hachage étant irréversible, le mot de passe n'est jamais déchiffré. Spring Security compare directement l'empreinte du mot de passe saisi avec le hash stocké.

G. CORS, validation et security headers

D'autres mécanismes complètent la sécurité de Music Wall.

1 - CORS

Le backend utilise une configuration CORS afin de définir l'origine autorisée à appeler l'API depuis un navigateur. Les méthodes HTTP nécessaires à l'application sont explicitement autorisées.

Cette configuration permet notamment au frontend Angular déployé séparément de communiquer avec le backend tout en évitant d'autoriser arbitrairement toutes les origines.

```
CorsConfiguration configuration = new CorsConfiguration();
configuration.setAllowedOrigins(List.of(corsOrigin));
configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
configuration.setAllowedHeaders(List.of("*"));
```

Figure 28 - Exemple : Configuration CORS - SecurityConfig.java

Le backend limite les origines autorisées et définit explicitement les méthodes HTTP acceptées.

2 - Validation

Les DTO reçus par l'API utilisent Bean Validation afin de contrôler certaines données avant leur traitement par la logique métier.

Par exemple, lors de l'inscription, le nom d'utilisateur est obligatoire et doit contenir entre 3 et 30 caractères. Le mot de passe est également obligatoire et doit contenir au minimum 8 caractères.

L'annotation @Valid utilisée dans les Controllers déclenche ces validations lors de la réception de la requête.

```
@NotBlank(message = "Username is required")
@Size(min = 3, max = 30, message =
    "Username must contain between 3 and 30 characters")
private String username;

@NotBlank(message = "Password is required")
@Size(min = 8, message =
    "Password must contain at least 8 characters")
private String password;
```

Figure 29 - Exemple : Validation des données - RegisterRequest.java

Les données reçues sont vérifiées avant leur traitement par la logique métier.

3 - Security headers

Des en-têtes HTTP de sécurité sont également configurés pour la version déployée de l'application.

X-Content-Type-Options limite l'interprétation incorrecte du type MIME d'une ressource.

X-Frame-Options empêche l'application d'être intégrée dans une frame, ce qui contribue à la protection contre le clickjacking. Referrer-Policy contrôle les informations transmises dans le header Referrer.

Permissions-Policy désactive des fonctionnalités du navigateur qui ne sont pas nécessaires à Music Wall, notamment la caméra, le microphone et la géolocalisation.

```
[headers.values]
  X-Content-Type-Options = "nosniff"
  X-Frame-Options = "DENY"
  Referrer-Policy = "strict-origin-when-cross-origin"
  Permissions-Policy = "camera=(), microphone=(), geolocation=()"
```

Figure 30 - Exemple : Security headers configurés pour le déploiement - netlify.toml

Ces en-têtes renforcent la sécurité de l'application au niveau du navigateur.

XI. Politique de tests

Des tests ont été mis en place afin de vérifier le comportement de plusieurs fonctionnalités importantes de Music Wall.

La stratégie de test distingue principalement les tests unitaires, qui vérifient une partie du code de manière isolée, et les tests d'intégration, qui vérifient le fonctionnement de plusieurs couches de l'application ensemble.

A. Tests unitaires

Les tests unitaires permettent de vérifier le comportement d'une classe ou d'une méthode de manière isolée.

Côté backend, Music Wall utilise JUnit 5 et Mockito. Les dépendances d'un service, comme les repositories ou d'autres services, peuvent être remplacées par des mocks afin de tester uniquement la logique métier concernée.

Ces tests sont notamment utilisés pour vérifier les règles liées aux murs musicaux et aux droits d'accès.

Par exemple, un test vérifie qu'un membre peut accéder à un mur partagé mais qu'il ne peut pas effectuer une opération réservée au propriétaire.

D'autres tests vérifient également qu'à la création d'un mur, son créateur devient bien propriétaire sans être ajouté une seconde fois dans la liste des membres.

Côté frontend, des tests Jasmine permettent également de vérifier certains comportements Angular. L'interceptor d'authentification est notamment testé afin de distinguer une réponse 401, qui entraîne la déconnexion, d'une réponse 403, qui indique simplement que l'utilisateur ne possède pas les droits nécessaires pour une action.

```
assertEquals(
    wall,
    service.findAccessibleWall(username: "alice", wallId: 10L));
assertThrows(
    expectedType: ForbiddenException.class, () ->
    service.findOwnedWall(username: "alice", wallId: 10L));
```

Figure 31 - Exemple : test unitaire - WallAccessServiceTest.java

Ce test vérifie qu'un membre peut consulter un mur partagé tout en restant interdit d'une opération réservée au propriétaire.

B. Tests d'intégration

Les tests d'intégration permettent de vérifier le fonctionnement conjoint de plusieurs composants de l'application.

Dans Music Wall, les tests d'intégration backend utilisent Spring Boot, MockMvc et un profil de test avec une base H2. MockMvc permet de simuler des requêtes HTTP vers l'API sans devoir lancer manuellement un navigateur.

Ces tests traversent plusieurs couches de l'application, notamment le Controller, le Service, Spring Security et le Repository.

Un test consacré à l'inscription vérifie par exemple qu'un mot de passe trop court est refusé, qu'une inscription valide retourne le statut HTTP attendu et un token JWT, puis que l'utilisateur est réellement enregistré avec un mot de passe hashé.

D'autres tests vérifient notamment le changement de mot de passe, la limite de longueur de la biographie du profil et la persistance des valeurs de wallpaper.

```
String responseBody = mockMvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {
                          "username": "integration_user",
                          "password": "password1"
                        }
                        """))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.username").value("integration_user"))
        .andExpect(jsonPath("$.token").isNotEmpty())
```

```
var savedUser =
userRepository.findByUsername(username: "integration_user").orElseThrow();
assertNotEquals(unexpected: "password1", savedUser.getPassword());
assertTrue(passwordEncoder.matches("password1", savedUser.getPassword()));
```

Figure 32 - Exemple : Test d'intégration - AuthIntegrationTest.java

Ce test simule une inscription réelle à travers l'API et vérifie à la fois la réponse HTTP, la génération du JWT et le stockage du mot de passe sous forme de hash.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.303 s
[INFO] Finished at: 2026-09-06T23:18:56+02:00
[INFO] ------------------------------------------------------------------------
```

Figure 33 - Exécution des tests automatisés du backend

L'exécution de la suite de tests confirme que les tests automatisés du backend sont exécutés avec succès, sans échec ni erreur.

C. Tests manuels de l'API avec Postman

En complément des tests automatisés, j'ai utilisé Postman pendant le développement afin de tester manuellement les endpoints de l'API REST. Postman m'a notamment permis d'envoyer des requêtes GET, POST, PUT et DELETE, de vérifier les codes de statut HTTP, les données JSON retournées par le backend ainsi que le comportement des endpoints protégés par authentification.

Pour les requêtes nécessitant une authentification, le token JWT obtenu après la connexion était transmis dans l'en-tête Authorization sous la forme Bearer <token>. Ces tests manuels ont été utiles pendant le développement pour vérifier rapidement le comportement de l'API avant ou en complément des tests automatisés.

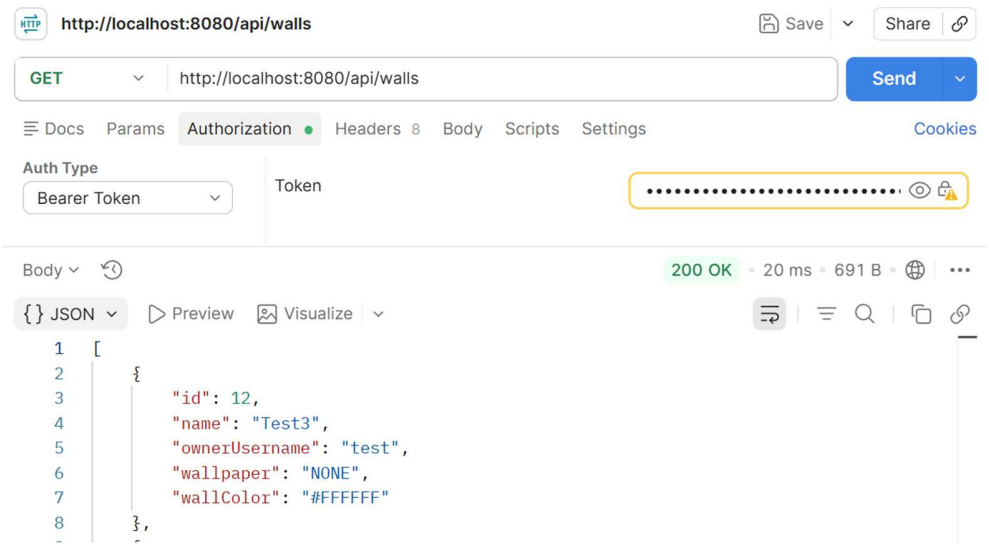


Figure 34 - Exemple de test manuel de l'API REST avec Postman

Cet exemple montre l'envoi d'une requête HTTP vers l'API et la vérification du statut ainsi que de la réponse JSON retournée par le backend.

XII. Déploiement

Music Wall peut être exécuté localement dans un environnement conteneurisé avec Docker et Docker Compose.

Pour la version accessible en ligne, le frontend Angular est déployé sur Netlify, tandis que le backend Spring Boot et la base PostgreSQL sont hébergés sur Render.

A. Docker

Docker permet d'encapsuler une application et ses dépendances dans une image afin de disposer d'un environnement reproductible.

Music Wall possède un Dockerfile pour le backend et un autre pour le frontend.

Le backend utilise une construction en plusieurs étapes. Une première image Maven compile le projet Spring Boot et produit le fichier JAR. Une seconde image Java plus légère est ensuite utilisée pour exécuter uniquement l'application compilée.

Le frontend utilise également une construction en plusieurs étapes. Une image Node.js compile l'application Angular, puis les fichiers statiques générés sont copiés dans une image Nginx qui les sert au navigateur.

Cette approche permet de séparer les outils nécessaires à la compilation de ceux nécessaires à l'exécution finale de l'application.

```
FROM maven:3.9.10-eclipse-temurin-17 AS build

WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:17-jre-alpine

WORKDIR /app
COPY --from=build /app/target/music-wall-backend-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figure 35 - Dockerfile backend

Le backend est d'abord compilé avec Maven, puis le fichier JAR obtenu est exécuté dans une image Java plus légère.

```
FROM node:22-alpine AS build

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:1.27-alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/music-wall-frontend/browser/ /usr/share/nginx/html/

EXPOSE 80
```

Figure 36 - Dockerfile frontend

Angular est compilé dans une image Node.js, puis le résultat de la compilation est servi par Nginx.

B. Docker Compose / Nginx

Docker Compose permet de lancer les différentes parties de Music Wall comme un ensemble cohérent.

La configuration locale comprend quatre services :

- postgres pour la base de données PostgreSQL ;

- backend pour l'API Spring Boot ;

- catalogue-seed pour charger le catalogue de référence dans une base vide ;

- frontend pour l'application Angular servie par Nginx.

Le backend attend que PostgreSQL soit disponible avant de démarrer. Le service de catalogue attend ensuite que le backend soit opérationnel avant de vérifier si les données du catalogue doivent être chargées.

Un volume Docker permet de conserver les données PostgreSQL même lorsque les conteneurs sont arrêtés.

Nginx sert les fichiers statiques produits par Angular. Il joue également le rôle de reverse proxy : les requêtes commençant par /api sont transférées vers le backend Spring Boot sur le réseau Docker interne.

Enfin, la règle try_files permet de rediriger les routes Angular vers index.html, ce qui est nécessaire pour le fonctionnement d'une Single Page Application.

```yaml
services:
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME} -d music_wall_rncp"]

  backend:
    build:
      context: ./backend
    depends_on:
      postgres:
        condition: service_healthy

  catalogue-seed:
    image: postgres:16-alpine
    depends_on:
      backend:
        condition: service_healthy

  frontend:
    build:
      context: ./frontend
    depends_on:
      catalogue-seed:
        condition: service_completed_successfully
```

Figure 37 – Exemple : Orchestration Docker Compose - docker-compose.yml

Cet extrait montre que Docker Compose orchestre les différents services de l'application et définit leur ordre de démarrage grâce aux dépendances et aux health checks.

```
location /api/ {
    proxy_pass http://backend:8080;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location / {
    try_files $uri $uri/ /index.html;
}
```

Figure 38 - Exemple : Nginx - nginx.conf

Nginx transmet les requêtes /api vers le backend Spring Boot et renvoie index.html pour les routes gérées par Angular.

C. Production Netlify / Render

Pour la mise en ligne de Music Wall, les différentes parties de l'application sont déployées séparément.

Le frontend Angular est hébergé sur Netlify. Lors du déploiement, Netlify exécute la commande de build Angular puis publie les fichiers statiques générés.

Le backend Spring Boot est hébergé sur Render et communique avec une base PostgreSQL également hébergée sur Render.

La configuration Netlify redirige les requêtes /api vers le backend déployé sur Render. Pour l'utilisateur, le frontend peut ainsi continuer à appeler des URLs commençant par /api sans avoir à manipuler directement l'adresse du serveur backend.

Les variables sensibles, notamment les identifiants de la base de données et le secret utilisé pour signer les JWT, sont fournies dans l'environnement de déploiement et ne sont pas intégrées directement dans le code source.

Le catalogue PostgreSQL de production a été initialisé séparément à partir du fichier catalogue_seed.sql. Le mécanisme automatique catalogue-seed de Docker Compose concerne l'environnement local et n'est pas utilisé comme mécanisme de démarrage de la production.

```
[build]
  base = "frontend"
  command = "npm run build"
  publish = "dist/music-wall-frontend/browser"

[[redirects]]
  from = "/api/*"
  to = "https://music-wall-backend.onrender.com/api/:splat"
  status = 200
  force = true
```

Figure 39 - Exemple: production Netlify / Render - netlify.toml

Cette configuration construit le frontend Angular sur Netlify et redirige les appels /api vers le backend Spring Boot hébergé sur Render.

XIII. Veille technologique

La veille technologique réalisée pendant le développement de Music Wall a principalement porté sur les technologies utilisées dans le projet et sur les problématiques rencontrées lors de leur mise en oeuvre. Les enseignements suivis à l'université m'ont apporté les bases nécessaires en développement web, Java, Spring Boot, bases de données et conception d'applications. En complément de ces cours, je me suis appuyée sur les documentations officielles des technologies utilisées, notamment Angular, Spring Boot, Spring Security, PostgreSQL et Docker.

J'ai également utilisé des tutoriels vidéo, notamment sur YouTube, afin de mieux comprendre certains concepts et leur mise en pratique. Ces ressources m'ont notamment aidée à approfondir la création d'une API REST avec Spring Boot, l'organisation entre Controller, Service et Repository ainsi que le traitement des requêtes HTTP. Lorsque je rencontrais un problème précis, je complétais ces ressources par des recherches ciblées afin de comparer différentes solutions avant de les appliquer au projet.

Un premier axe de recherche a concerné la sécurité d'une API REST. J'ai notamment étudié le fonctionnement de Spring Security, de l'authentification stateless avec JWT, du stockage des mots de passe avec BCrypt, de CORS ainsi que les principales protections contre les attaques XSS, CSRF et les injections SQL.

Un deuxième axe a concerné la recherche dans le catalogue musical. PostgreSQL est utilisé avec l'extension pg_trgm et des index GIN afin d'améliorer les recherches textuelles sur les noms d'artistes et les titres d'albums ou de morceaux.

La gestion du catalogue a également nécessité des recherches sur MusicBrainz. Cette source a finalement été utilisée uniquement pour préparer les données initiales du catalogue à l'aide d'un outil d'import séparé. MusicBrainz n'est donc pas une dépendance nécessaire au fonctionnement normal de l'application.

Enfin, la mise en ligne du projet m'a amenée à travailler sur la conteneurisation avec Docker et Docker Compose ainsi que sur le déploiement séparé du frontend Angular sur Netlify et du backend Spring Boot avec PostgreSQL sur Render.

Cette veille a donc été directement liée aux besoins du projet. Elle m'a permis de consolider les connaissances acquises pendant la formation, de mieux comprendre le fonctionnement des outils utilisés et de comparer différentes solutions afin de retenir celles qui correspondaient le mieux à l'architecture de Music Wall.

XIV. Difficultés rencontrées

A - Gestion des droits d'accès

Une première difficulté a concerné la distinction entre authentification et autorisation.

Il ne suffisait pas de vérifier qu'un utilisateur était connecté. L'application devait également déterminer s'il était propriétaire d'un mur, membre de celui-ci ou utilisateur extérieur.

J'ai donc centralisé une partie de ces vérifications dans le WallAccessService. Un propriétaire peut modifier les paramètres du mur et gérer ses membres, tandis qu'un membre peut accéder au mur et gérer son contenu sans disposer des opérations réservées au propriétaire.

Cette organisation m'a également permis de mieux distinguer les réponses HTTP 401, liées à l'authentification, et 403, liées à un manque d'autorisation.

Cette gestion des autorisations est illustrée dans la partie consacrée à la sécurité.

B - Gestion du catalogue musical

La gestion du catalogue musical a également constitué une difficulté importante.

Une première version du projet dépendait davantage d'une source musicale externe. Cette solution augmentait la complexité de l'application et créait une dépendance au réseau et au service externe.

J'ai finalement choisi de conserver un catalogue local dans PostgreSQL. MusicBrainz est utilisé uniquement par un outil séparé pour préparer les données initiales, qui sont ensuite importées dans la base.

Le fonctionnement normal de Music Wall reste ainsi simple : Angular, Spring Boot, PostgreSQL.

Cette évolution a permis de réduire les dépendances externes et de conserver un comportement prévisible du catalogue.

C - Modélisation des relations

Certaines relations de données ont demandé une attention particulière lors de la conception.

La gestion des utilisateurs d'un mur comprend par exemple deux relations différentes : un mur possède un propriétaire unique, mais peut également posséder plusieurs membres. Le propriétaire n'est pas ajouté une seconde fois dans la liste des membres.

Le catalogue contient également plusieurs relations plusieurs-à-plusieurs, notamment entre les albums et les genres ainsi qu'entre les morceaux et les genres. Elles sont représentées dans la base par des tables d'association.

Enfin, un élément ajouté à un mur doit référencer soit un morceau, soit un album. Les deux références sont facultatives au niveau de la structure de la base, mais une règle métier impose qu'une seule des deux soit renseignée.

La réalisation du MCD, du MLD puis du MPD m'a permis de clarifier progressivement ces relations.

D - Déploiement de l'application

Le passage de l'environnement local à la version en ligne a constitué une autre difficulté.

En local, Docker Compose permet de lancer le frontend, le backend et PostgreSQL dans un même environnement. En production, ces éléments sont au contraire déployés séparément : Angular sur Netlify et Spring Boot avec PostgreSQL sur Render.

Cette séparation a nécessité d'adapter la communication entre le frontend et le backend, notamment la configuration des URLs de l'API et de CORS.

Le catalogue de la base PostgreSQL de production a également dû être initialisé séparément, car le service catalogue-seed de Docker Compose est destiné à l'environnement local.

Ce travail m'a permis de mieux comprendre la différence entre l'environnement de développement, la conteneurisation locale et un déploiement réel sur plusieurs services.

XV. Conclusion / améliorations futures

Le développement de Music Wall m'a permis de réaliser une application web complète reposant sur une architecture séparant le frontend Angular, le backend Spring Boot et une base de données PostgreSQL.

Le projet permet aujourd'hui à un utilisateur de créer un compte, de s'authentifier, de gérer son profil, de créer des murs musicaux, d'organiser leur contenu en sections, d'ajouter des albums ou des morceaux depuis un catalogue local et de partager un mur avec d'autres utilisateurs enregistrés.

La réalisation du projet m'a permis de mettre en pratique plusieurs compétences abordées pendant la formation : conception d'une base de données relationnelle, développement d'une interface Angular, création d'une API REST, utilisation de Spring Data JPA, mise en place de l'authentification et des autorisations, réalisation de tests et déploiement d'une application.

Elle m'a également permis de mieux comprendre l'importance de la séparation des responsabilités entre les différentes couches de l'application et de la conception du modèle de données avant l'implémentation.

Music Wall reste néanmoins une première version qui pourrait être enrichie dans le futur.

Parmi les évolutions possibles figurent un système d'amis et de demandes d'amis, un système d'invitations avant l'ajout à un mur, des rôles plus détaillés pour les membres, la gestion de favoris, davantage de paramètres de confidentialité pour les profils, des statistiques basées sur les genres musicaux ainsi qu'une fonctionnalité de découverte de concerts.

D'autres améliorations pourraient également concerner la sécurité, les tests automatisés et l'évolution de l'interface utilisateur.

Ces fonctionnalités n'ont volontairement pas été intégrées dans la version actuelle afin de conserver un périmètre cohérent et de finaliser une application fonctionnelle, testable et déployable.

XVI. Remerciements

Je tiens à remercier l'ensemble de l'équipe de DesCodeuses pour l'accompagnement proposé tout au long de cette formation et pour l'opportunité de développer mes compétences dans le domaine de l'informatique. Je remercie tout particulièrement Émilie et Tarek, nos formateurs, pour leur disponibilité, leurs explications, leurs conseils et leur accompagnement pendant la formation. Je remercie également Kenza, qui nous a accompagnées sur la dernière partie du parcours.

Enfin, je souhaite remercier ma famille pour son soutien, ses encouragements et sa patience tout au long de cette période de formation et de réalisation de mon projet.